ARTEFACTOS DEL MÓDULO

Punto de Control 3 (PC3)

Fase de Elaboración: arquitectura, contratos y diseño del módulo

Pacíficos Online — IF6100

INFORMACIÓN DEL SUBGRUPO

Subgrupo y módulo	*SG2_Micrositios_y_Administración_De_Comercios*
Integrantes	Diego Alfaro Alvarado C4C207 Abril Chacón Araya C4E107 Mia Wilson C4L119 Josefh Argüello Ramos C30675 Jorge M. Salazar Navarro C4J653
Representante ante el Comité	Diego Alfaro Alvarado
Fecha de entrega	Jueves 22 de octubre de 2026 (Semana 11)
Versión del documento	*[0.1 — borrador]*

1. Dependencias del módulo

Antes de diseñar, el subgrupo deja claro qué necesita de otros módulos, qué les ofrece y qué toma de los artefactos del Comité. Esta sección alimenta el contrato (sección 2), los diagramas de interacción (sección 5) y los dobles de prueba (sección 9).

1.1 Dependencias con los artefactos del Comité (PC3)

Cada artefacto del subgrupo usa algo que define el Comité. Si el Comité aún no lo ha publicado, el subgrupo anota el supuesto que usa y lo ajusta cuando esté disponible.

Artefacto del subgrupo	Depende de (Comité PC3)	Qué toma de ahí	Estado
2. Contrato OpenAPI	4.1 Reglas de publicación de contratos; carpeta contratos/ del repositorio; 5.5 Manejo de errores	Formato, esquema de versionado, ubicación del archivo, estructura estándar de error	Disponible
3 y 4. Realización y clases de diseño	1.5 Vista lógica (capas); 5. Núcleo compartido	Capas internas que siguen todos los módulos; clases del núcleo para identidad, roles y aislamiento	Disponible
5. Diagramas de interacción	4.2 y 4.3 Contratos de otros módulos; 1.6 Vista de procesos	Operaciones y eventos disponibles; qué interacciones son síncronas o por eventos	Disponible con supuestos
6. Modelo de datos y migraciones	6. Modelo de datos consolidado; 1.8 Convenciones de implementación; 1.9 Decisiones de datos	Tablas reservadas, claves compartidas, convención de nombres, uso o no de llaves foráneas entre módulos	Disponible
7. Patrones de diseño	3. ADR	Decisiones transversales que los patrones del módulo no deben contradecir	Disponible
8. Prototipo de interfaz	ADR del frontend; carpeta frontend/ y componentes compartidos	React + Tailwind CSS, organización por módulo, componentes comunes	Disponible
9. Dobles de prueba	4. Contratos publicados de otros módulos	Versión del contrato que el doble debe imitar	Disponible con supuestos

1. Lo que el subgrupo entrega al Comité

El Comité necesita estos insumos para cerrar sus artefactos. El representante ante el Comité es el responsable de entregarlos.

Insumo	Destino en el Comité	Fecha	Entregado
Lista de tablas corregida del módulo	6. Modelo de datos consolidado	Jue 1 oct	sí
Borrador del contrato (operaciones y eventos)	4. Contratos publicados	Sáb 3 oct	Sí
Contrato OpenAPI en la carpeta contratos/	4. Contratos publicados	Pendiente de confirmar	No
Módulo desplegado en el ambiente (implementación mínima)	7. Ambiente de integración	Pendiente	No

2. Contrato formal del módulo (OpenAPI)

Descripción formal de las operaciones y eventos que el módulo ofrece, versionada y publicada en el repositorio.

2.1 Datos del contrato

Versión	v1.2.1
Archivo en el repositorio	contracts/m2-comercios/v1/openapi.yaml
Versión de OpenAPI	3.0.3
Estado	publicado
Consumidores	Frontend, M1, M3, M4, M5, M6, M7

2.2 Operaciones

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/api/m2/v1/comercios/publico	Obtiene la lista de comercios actualizados para el portal público. Permite filtrar los destacados	Query: pagina (int, ≥1, defecto 1), por_pagina (int, 1–50, defecto 20), destacado (bool, defecto false).	200 - ListadoComercios	422 - ERR-005
POST	/api/m2/v1/comercios/publico/lote	Obtiene el resumen de varios comercios habilitados en una sola solicitud.	Body SolicitudLoteComercios: ids_comercio, arreglo de 1 a 50 enteros únicos.	200 — ResumenComerciosLote	422 — ERR-005
GET	/api/m2/v1/comercios/{id_comercio}/publico	Obtiene la información pública de un comercio para mostrar su micrositio. Solo devuelve comercios Activos.	Path: id_comercio (int).	200 — ComercioPublico	404 — ERR-004; 422 — ERR-005
GET	/api/m2/v1/comercios/{id_comercio}/estado	Devuelve el estado operativo del comercio.	Path: id_comercio (int).	200 — EstadoComercio	401 ERR-001; 403 ERR-002;

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
		Consumida por M3 y M4			404 ERR-003/ERR-004; 422 ERR-005
GET	/api/m2/v1/comercios/{id_comercio}/sedes	Obtiene las sedes físicas habilitadas del comercio para que M4 valide sus ubicaciones.	Path: id_comercio (int).	200 — SedesComercio	401 ERR-001; 403 ERR-002; 404 ERR-003/ERR-004; 422 ERR-005
GET	/api/m2/v1/comercios/{id_comercio}/politicas-envio	Devuelve políticas y tarifas de envío del comercio para M5.	Path: id_comercio (int).	200 — PoliticasEnvio	401 ERR-001; 403 ERR-002; 404 ERR-003/ERR-004; 422 ERR-005
GET	/api/m2/v1/comercios/{id_comercio}/datos-fiscales	Devuelve datos fiscales y configuración de comisión para M6 y M7.	Path: id_comercio (int).	200 — DatosFiscalesComercio	401 ERR-001; 403 ERR-002; 404 ERR-003/ERR-004; 422 ERR-005
GET	/api/m2/v1/subpedidos/{id_subpedido}/comision	Devuelve la comisión ya calculada por M2 para el subpedido. Es consumida por M7.	Path: id_subpedido (int).	200 — ComisionSubpedido	401 ERR-001; 403 ERR-002; 404 ERR-003/ERR-004; 422 ERR-005

2.3 Eventos publicados

Evento	Versión	Cuándo se publica	Carga útil (campos y tipos)
ComercioAprobado	v1	Después de confirmar la aprobación y guardar el estado en comercio “Activo”	id_evento: UUID, fecha_hora: date-time, id_comercio: integer, estado: "Activo", nombre_comercial: string
ComercioSuspendido	v1	Después de confirmar la suspensión y guardar el motivo y el estado “Suspendido”	id_evento: UUID, fecha_hora: date-time, id_comercio: integer, estado: "Suspendido", motivo: string
ComercioReactivado	v1	Después de confirmar la reactivación de un comercio suspendido y devolverlo a estado Activo	id_evento: UUID, fecha_hora: date-time, id_comercio: integer, estado: "Suspendido", motivo: string

2.4 Esquemas expuestos

Esquema	Campos expuestos
ErrorEstandar	error { codigo:string, mensaje:string, detalles:object
ComercioPublico	id_comercio:int, nombre_comercial:string, `descripcion:string
ImagenGaleria	id_imagen:int, imagen_url:uri, texto_alternativo:string
ContactoPublico	telefono:string
BannerPublico	id_banner:int, imagen_url:uri, texto_alternativo:string
HorarioPublico	id_horario:int, dia_semana:int (1–7), hora_apertura, hora_cierre, cerrado:boolean
RedSocialPublica	plataforma:string, url:uri
PoliticaPublica	tipo:string, descripcion:string

Esquema	Campos expuestos
ComercioResumen	id_comercio:int, nombre_comercial:string, `logo_url:uri
SolicitudLoteComercios	ids_comercio:int[] — mínimo 1, máximo 50, sin duplicados
ResumenComerciosLote	comercios:ComercioResumen[]
ListadoComercios	comercios:ComercioResumen[]
EstadoComercio	id_comercio:int, estado:{Pendiente, Activo, Rechazado, Suspendido}, activo:boolean, motivo_estado:string
SedesComercio	id_comercio:int, sedes:SedeHabilitada[]
SedeHabilitada	id_sede:int, nombre:string, direccion:string, habilitada:boolean
PoliticasEnvio	id_comercio:int, politicas:PoliticaPublica[], tarifas:TarifaEnvio[]
TarifaEnvio	zona:string, costo:double, moneda:string
DatosFiscalesComercio	id_comercio:int, cedula_juridica:string, razon_social:string, direccion_fiscal:string, inscrito_como_contribuyente:boolean, comision:ConfiguracionComision
ConfiguracionComision	tipo:string (porcentajelmonto_fijo), valor:double, base_calculo:string, `moneda:string
ComisionSubpedido	id_subpedido:int, id_comercio:int, monto_comision:double, base_utilizada:double, moneda:string

2.5 Historial de versiones del contrato

Versión	Fecha	Cambio	¿Rompe consumidores?
v1.2.1	Pendiente de confirmar	Versión actual del contrato formal de M2. Define las operaciones, esquemas, eventos publicados y dependencias consumidas del módulo	No

3. Realización de casos de uso (mínimo dos)

Muestra cómo las clases del módulo cumplen un caso de uso, separando clases límite, control y entidad.

3.1 Realización del caso de uso [CU-0015 - Aprobar solicitud de comercio]

Caso de uso (PC2)	*CU-0015 -Aprobar solicitud de comercio*
Actor principal	*Encargado de la plataforma*
Módulos externos involucrados	*Núcleo: identidad y auditoría. M1, M3 y M4: consumidores de ComercioAprobado*

Estereotipo	***Clase***	***Responsabilidad en este caso de uso***
Límite	*SolicitudesController*	*Recibe la petición de aprobación, valida los datos de entrada, solicita autorización y delega la operación en ComerciosService.*
Control — Policy	*SolicitudComercioPolicy*	*Comprueba que el usuario tenga permiso para aprobar la solicitud, utilizando VerificaRol del núcleo.*
Control	*ComerciosService*	*Coordina la aprobación. Comprueba la solicitud pendiente, la información empresarial y la condición de contribuyente; verifica los documentos cuando corresponde, activa el comercio y solicita la creación de la cuenta.*
Entidad	*SolicitudComercio*	*Conserva la solicitud de incorporación, permite comprobar si está pendiente y registra su aprobación.*
Entidad	*Comercio*	*Conserva los datos del comercio y cambia su estado a Activo cuando se aprueba la solicitud.*
Entidad	*DocumentoComercio*	*Conserva los documentos del comercio y permite comprobar su vigencia y resultado de evaluación.*
Servicio del núcleo	*ServicioIdentidad*	*Crea la cuenta del encargado asociada al comercio aprobado y gestiona la invitación para establecer la contraseña.*
Servicio del núcleo	*BitacoraAuditoria*	*Registra la aprobación del comercio con el usuario responsable, la fecha, la acción y el resultado.*
Evento	*ComercioAprobado*	*Comunica la aprobación a M1, M3 y M4. Incluye id_evento, fecha_hora, id_comercio, estado y nombre_comercial. Se publica después del commit.*

Diagrama de Clases:

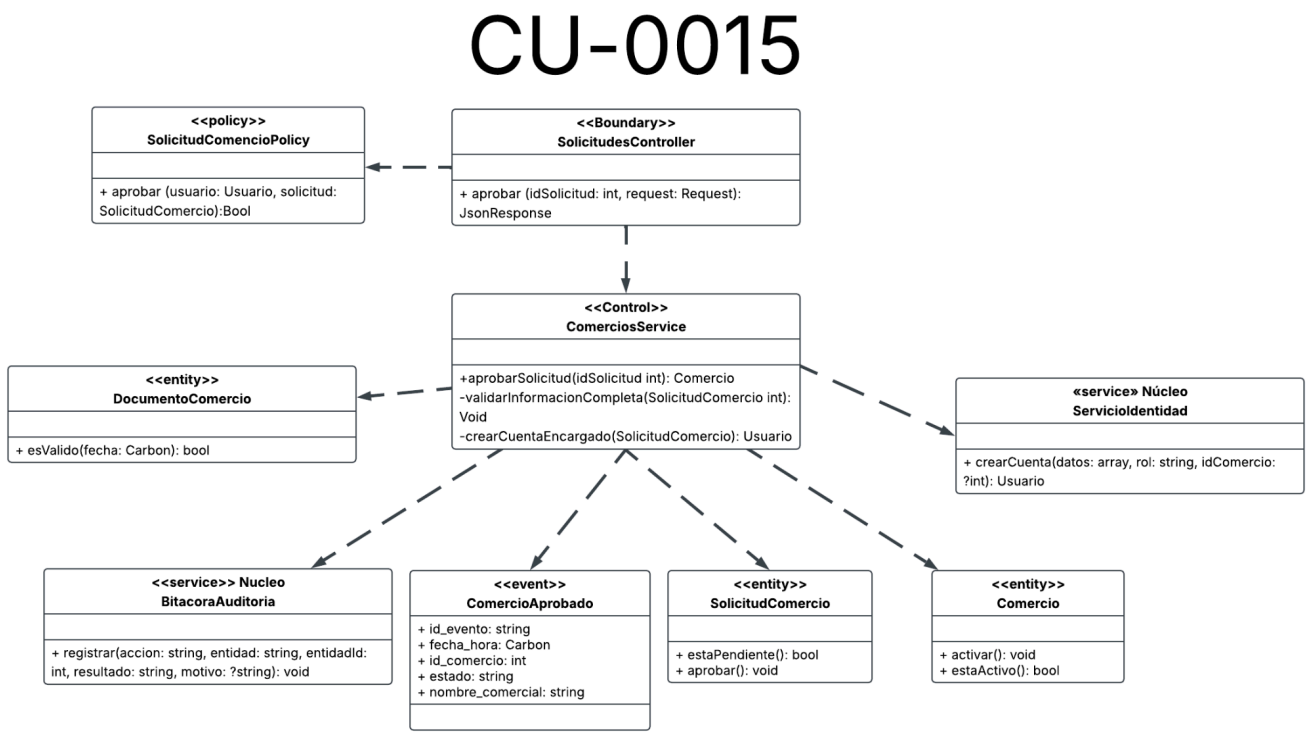
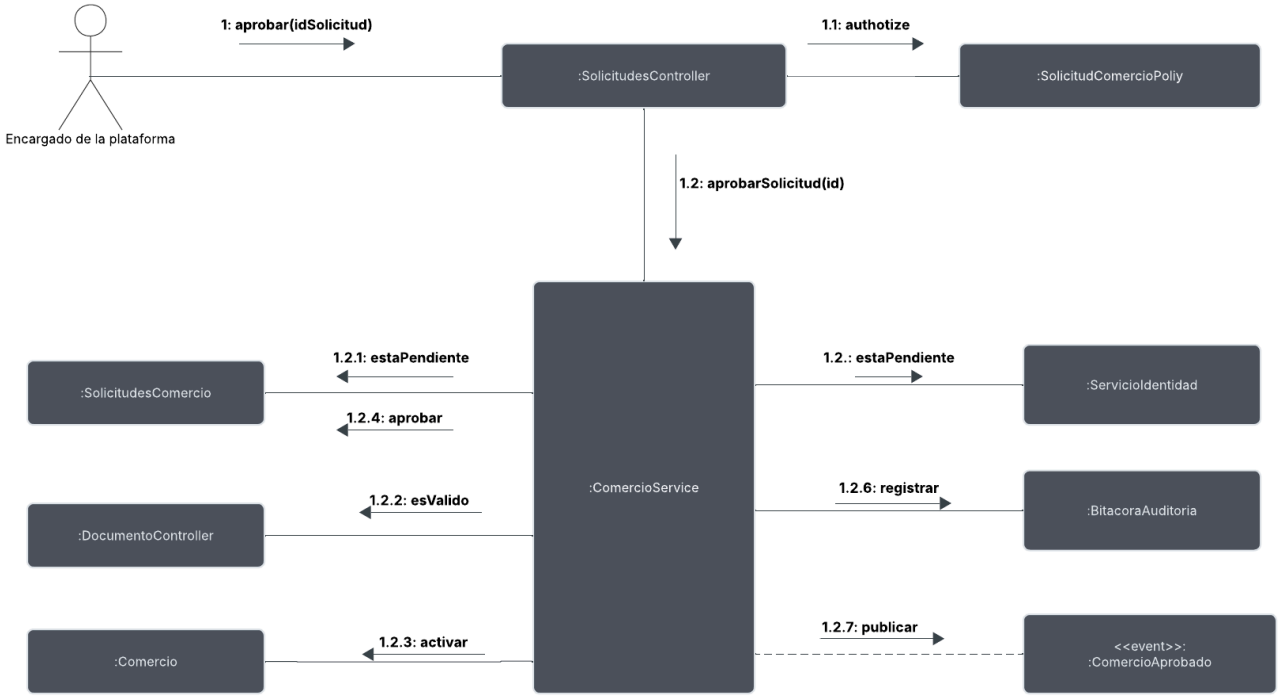


Diagrama de comunicación :



3.2 Realización del caso de uso [CU-0017 - Suspender comercio]

Caso de uso (PC2)	*CU-0017 - Suspender comercio*
Actor principal	*Encargado de la plataforma*
Módulos externos involucrados	Núcleo: auditoría. M1, M3, M4 y M5: consumidores de ComercioSuspendido

Estereotipo	*Clase*	*Responsabilidad en este caso de uso*
Límite	*ComerciosController*	*Recibe la petición de suspensión y el motivo, valida los datos de entrada, solicita autorización y delega la operación.*
Control — Policy	*ComercioPolicy*	*Comprueba que el usuario tenga permiso para suspender el comercio, utilizando VerificaRol del núcleo.*
Control	*ComerciosService*	*Comprueba que el comercio esté Activo y que el motivo no esté vacío. Coordina el cambio de estado, el historial de suspensión, la auditoría y la publicación del evento.*
Entidad	*Comercio*	*Permite comprobar el estado actual y cambiarlo a Suspendido.*
Entidad	*SuspensionComercio*	*Conserva el comercio afectado, el usuario responsable, el motivo y la fecha de suspensión.*
Servicio del núcleo	*BitacoraAuditoria*	*Registra la suspensión del comercio con el contexto del usuario, el comercio, la acción y el resultado.*
Evento	*ComercioSuspendido*	*Comunica la suspensión a M1, M3, M4 y M5. Incluye id_evento, fecha_hora, id_comercio, estado y motivo. Se publica después del commit.*

Diagrama de clases:

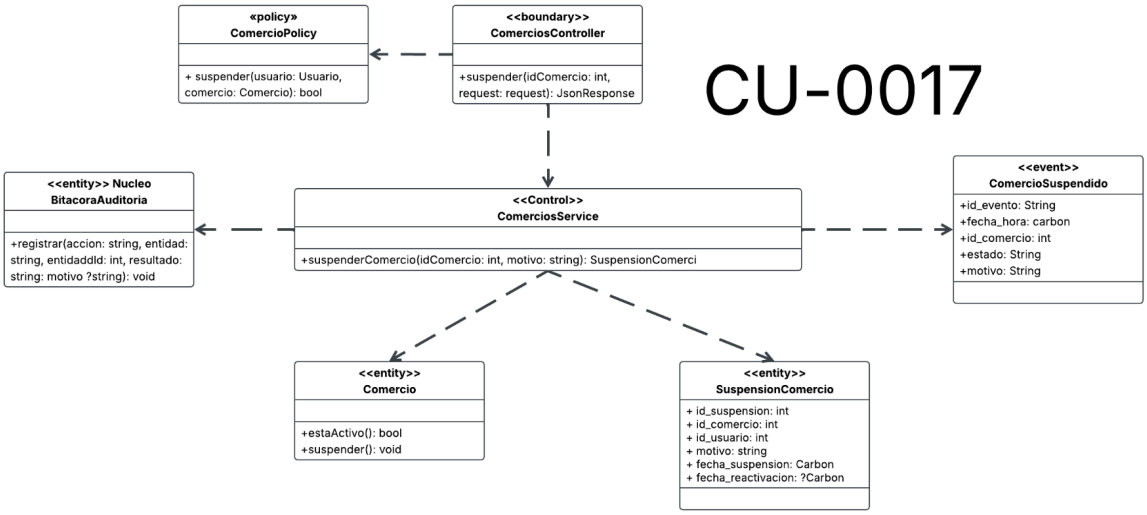
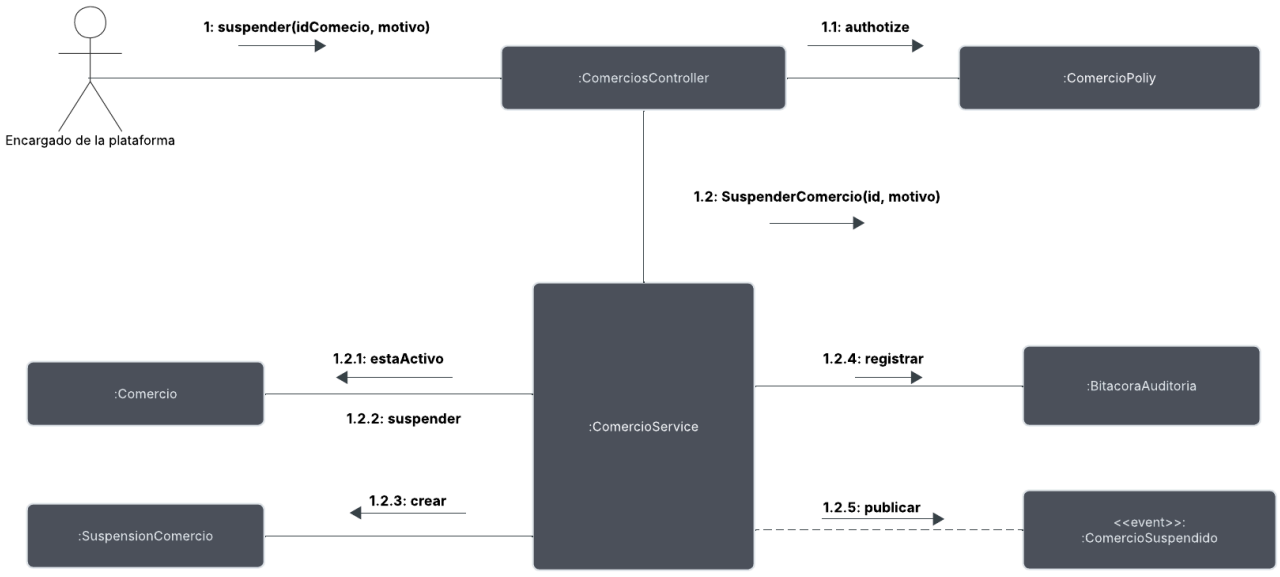
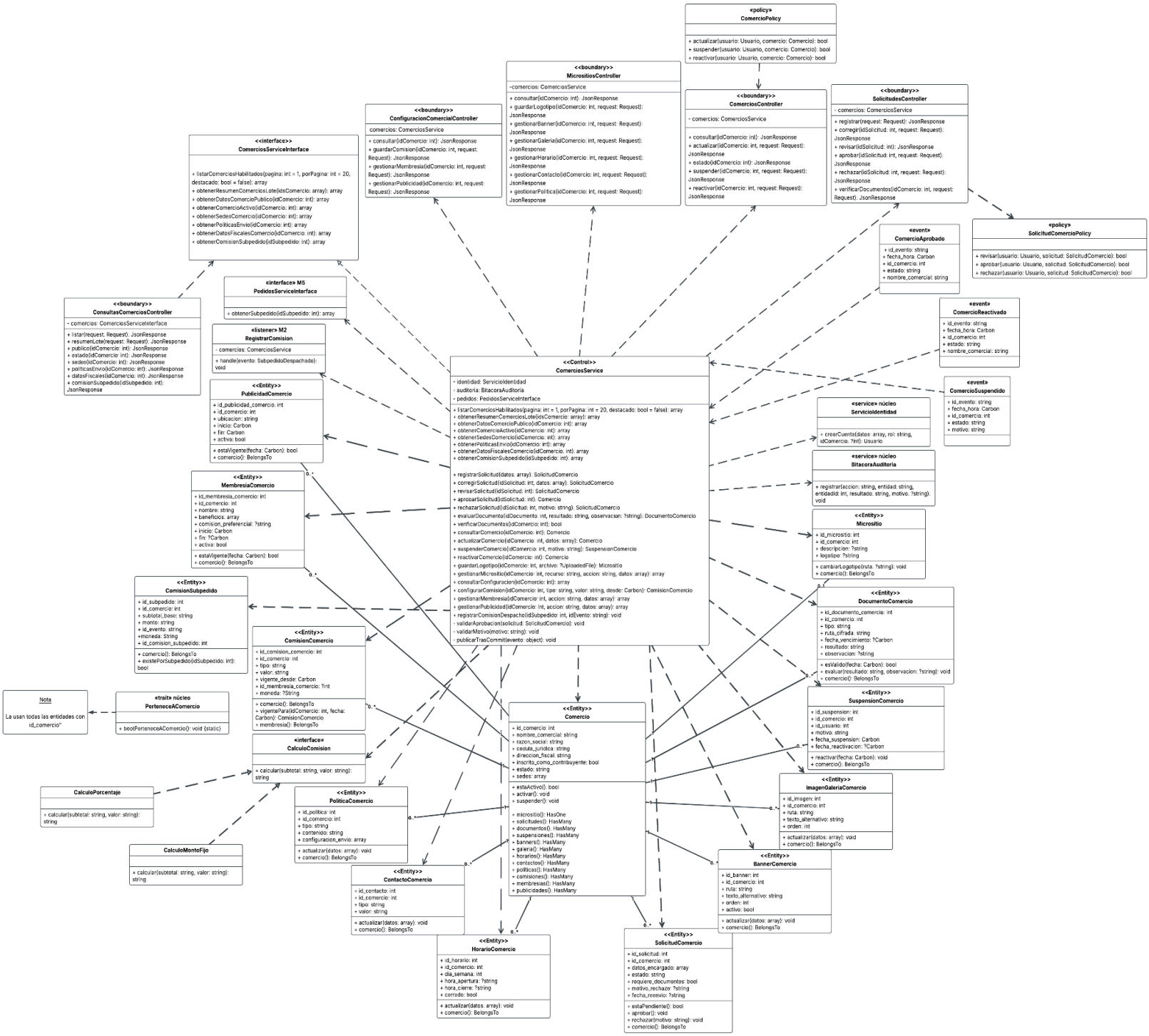

«policy»
ComercioPolicy
+ suspender(usuario: Usuario, comercio: Comercio): bool
<<boundary>>
ComerciosController
+suspender(idComercio: int, request: request): JsonResponse
CU-0017
<<entity>> Nucleo BitacoraAuditoria
+registrar(accion: string, entidad: string, entidaddId: int, resultado: string: motivo ?string): void
<<Control>>
ComerciosService
+suspenderComercio(idComercio: int, motivo: string): SuspensionComerci
<<event>>
ComercioSuspendido
+id_evento: String
+fecha_hora: carbon
+id_comercio: int
+estado: String
+motivo: String
<<entity>>
Comercio
+estaActivo(): bool
+suspender(): void
<<entity>>
SuspensionComercio
+ id_suspension: int
+ id_comercio: int
+ id_usuario: int
+ motivo: string
+ fecha_suspension: Carbon
+ fecha_reactivacion: ?Carbon

Diagrama de comunicación :



4. Diagrama de clases de diseño del módulo
Diagrama completo del módulo listo para programar: con tipos, visibilidad, firmas de métodos y navegabilidad.


«policy»
ComercioPolicy
+ actualizar(usuario: Usuario, comercio: Comercio): bool
+ suspender(usuario: Usuario, comercio: Comercio): bool
+ reactivar(usuario: Usuario, comercio: Comercio): bool
<<boundary>>
MicrositiosController
-comercios: ComerciosService
+ consultar(idComercio: int): JsonResponse
+ guardarLogotipo(idComercio: int, request: Request): JsonResponse
+ gestionarBanner(idComercio: int, request: Request): JsonResponse
+ gestionarGaleria(idComercio: int, request: Request): JsonResponse
+ gestionarHorario(idComercio: int, request: Request): JsonResponse
+ gestionarContacto(idComercio: int, request: Request): JsonResponse
+ gestionarPolitica(idComercio: int, request: Request): JsonResponse
<<boundary>>
ComerciosController
- comercios: ComerciosService
+ consultar(idComercio: int): JsonResponse
+ actualizar(idComercio: int, request: Request): JsonResponse
+ estado(idComercio: int): JsonResponse
+ suspender(idComercio: int, request: Request): JsonResponse
+ reactivar(idComercio: int, request: Request): JsonResponse
<<boundary>>
SolicitudesController
- comercios: ComerciosService
+ registrar(request: Request): JsonResponse
+ corregir(idSolicitud: int, request: Request): JsonResponse
+ revisar(idSolicitud: int): JsonResponse
+ aprobar(idSolicitud: int, request: Request): JsonResponse
+ rechazar(idSolicitud: int, request: Request): JsonResponse
+ verificarDocumentos(idComercio: int, request: Request): JsonResponse
<<boundary>>
ConfiguracionComercialController
comercios: ComerciosService
+ consultar(idComercio: int): JsonResponse
+ guardarComision(idComercio: int, request: Request): JsonResponse
+ gestionarMembresia(idComercio: int, request: Request): JsonResponse
+ gestionarPublicidad(idComercio: int, request: Request): JsonResponse
<<interface>>
ComerciosServiceInterface
+ listarComerciosHabilitados(pagina: int = 1, porPagina: int = 20, destacado: bool = false): array
+ obtenerResumenComerciosLote(idsComercio: array): array
+ obtenerDatosComercioPublico(idComercio: int): array
+ obtenerComercioActivo(idComercio: int): array
+ obtenerSedesComercio(idComercio: int): array
+ obtenerPoliticasEnvio(idComercio: int): array
+ obtenerDatosFiscalesComercio(idComercio: int): array
+ obtenerComisionSubpedido(idSubpedido: int): array
«event»
ComercioAprobado
+ id_evento: string
+ fecha_hora: Carbon
+ id_comercio: int
+ estado: string
+ nombre_comercial: string
«policy»
SolicitudComercioPolicy
+ revisar(usuario: Usuario, solicitud: SolicitudComercio): bool
+ aprobar(usuario: Usuario, solicitud: SolicitudComercio): bool
+ rechazar(usuario: Usuario, solicitud: SolicitudComercio): bool
«interface» M5
PedidosServiceInterface
+ obtenerSubpedido(idSubpedido: int): array
<<boundary>>
ConsultasComerciosController
- comercios: ComerciosServiceInterface
+ listar(request: Request): JsonResponse
+ resumenLote(request: Request): JsonResponse
+ publico(idComercio: int): JsonResponse
+ estado(idComercio: int): JsonResponse
+ sedes(idComercio: int): JsonResponse
+ politicasEnvio(idComercio: int): JsonResponse
+ datosFiscales(idComercio: int): JsonResponse
+ comisionSubpedido(idSubpedido: int): JsonResponse
«listener» M2
RegistrarComision
- comercios: ComerciosService
+ handle(evento: SubpedidoDespachado): void
«event»
ComercioReactivado
+ id_evento: string
+ fecha_hora: Carbon
+ id_comercio: int
+ estado: string
+ nombre_comercial: string
«event»
ComercioSuspendido
+ id_evento: string
+ fecha_hora: Carbon
+ id_comercio: int
+ estado: string
+ motivo: string
<<Entity>>
PublicidadComercio
+ id_publicidad_comercio: int
+ id_comercio: int
+ ubicacion: string
+ inicio: Carbon
+ fin: Carbon
+ activa: bool
+ estaVigente(fecha: Carbon): bool
+ comercio(): BelongsTo
<<Control>>
ComerciosService
- identidad: ServicioIdentidad
- auditoria: BitacoraAuditoria
- pedidos: PedidosServiceInterface
+ listarComerciosHabilitados(pagina: int = 1, porPagina: int = 20, destacado: bool = false): array
+ obtenerResumenComerciosLote(idsComercio: array): array
+ obtenerDatosComercioPublico(idComercio: int): array
+ obtenerComercioActivo(idComercio: int): array
+ obtenerSedesComercio(idComercio: int): array
+ obtenerPoliticasEnvio(idComercio: int): array
+ obtenerDatosFiscalesComercio(idComercio: int): array
+ obtenerComisionSubpedido(idSubpedido: int): array
+ registrarSolicitud(datos: array): SolicitudComercio
+ corregirSolicitud(idSolicitud: int, datos: array): SolicitudComercio
+ revisarSolicitud(idSolicitud: int): SolicitudComercio
+ aprobarSolicitud(idSolicitud: int): Comercio
+ rechazarSolicitud(idSolicitud: int, motivo: string): SolicitudComercio
+ evaluarDocumento(idDocumento: int, resultado: string, observacion: ?string): DocumentoComercio
+ verificarDocumentos(idComercio: int): bool
+ consultarComercio(idComercio: int): Comercio
+ actualizarComercio(idComercio: int, datos: array): Comercio
+ suspenderComercio(idComercio: int, motivo: string): SuspensionComercio
+ reactivarComercio(idComercio: int): Comercio
+ guardarLogotipo(idComercio: int, archivo: *UploadedFile): Micrositio
+ gestionarMicrositio(idComercio: int, recurso: string, accion: string, datos: array): array
+ consultarConfiguracion(idComercio: int): array
+ configurarComision(idComercio: int, tipo: string, valor: string, desde: Carbon): ComisionComercio
+ gestionarMembresia(idComercio: int, accion: string, datos: array): array
+ gestionarPublicidad(idComercio: int, accion: string, datos: array): array
+ registrarComisionDespacho(idSubpedido: int, idEvento: string): void
- validarAprobacion(solicitud: SolicitudComercio): void
- validarMotivo(motivo: string): void
- publicarTrasCommit(evento: object): void
«service» núcleo
ServicioIdentidad
+ crearCuenta(datos: array, rol: string, idComercio: ?int): Usuario
«service» núcleo
BitacoraAuditoria
+ registrar(accion: string, entidad: string, entidadId: int, resultado: string, motivo: ?string): void
<<Entity>>
MembresiaComercio
+ id_membresia_comercio: int
+ id_comercio: int
+ nombre: string
+ beneficios: array
+ comision_preferencial: ?string
+ inicio: Carbon
+ fin: ?Carbon
+ activa: bool
+ estaVigente(fecha: Carbon): bool
+ comercio(): BelongsTo
<<Entity>>
Micrositio
+ id_micrositio: int
+ id_comercio: int
+ descripcion: ?string
+ logotipo: ?string
+ cambiarLogotipo(ruta: ?string): void
+ comercio(): BelongsTo
<<Entity>>
ComisionSubpedido
+ id_subpedido: int
+ id_comercio: int
+ subtotal_base: string
+ monto: string
+ id_evento: string
+moneda: String
+ id_comision_subpedido: int
+ comercio(): BelongsTo
+ existePorSubpedido(idSubpedido: int): bool
<<Entity>>
DocumentoComercio
+ id_documento_comercio: int
+ id_comercio: int
- tipo: string
+ ruta_cifrada: string
+ fecha_vencimiento: ?Carbon
+ resultado: string
+ observacion: ?string
+ esValido(fecha: Carbon): bool
+ evaluar(resultado: string, observacion: ?string): void
+ comercio(): BelongsTo
<<Entity>>
ComisionComercio
+ id_comision_comercio: int
+ id_comercio: int
+ tipo: string
+ valor: string
+ vigente_desde: Carbon
+ id_membresia_comercio: ?int
+ moneda: ?String
+ comercio(): BelongsTo
+ vigentePara(idComercio: int, fecha: Carbon): ComisionComercio
+ membresia(): BelongsTo
<<Entity>>
SuspensionComercio
+ id_suspension: int
+ id_comercio: int
+ id_usuario: int
+ motivo: string
+ fecha_suspension: Carbon
+ fecha_reactivacion: ?Carbon
+ reactivar(fecha: Carbon): void
+ comercio(): BelongsTo
Nota
La usan todas las entidades con id_comercio"
«trait» núcleo
PerteneceAComercio
+ bootPerteneceAComercio(): void {static}
<<Entity>>
Comercio
+ id_comercio: int
+ nombre_comercial: string
+ razon_social: string
+ cedula_juridica: string
+ direccion_fiscal: string
+ inscrito_como_contribuyente: bool
+ estado: string
+ sedes: array
+ estaActivo(): bool
+ activar(): void
+ suspender(): void
+ micrositio(): HasOne
+ solicitudes(): HasMany
+ documentos(): HasMany
+ suspensiones(): HasMany
+ banners(): HasMany
+ galeria(): HasMany
+ horarios(): HasMany
+ contactos(): HasMany
+ politicas(): HasMany
+ comisiones(): HasMany
+ membresias(): HasMany
+ publicidades(): HasMany
«interface»
CalculoComision
+ calcular(subtotal: string, valor: string): string
<<Entity>>
ImagenGaleriaComercio
+ id_imagen: int
+ id_comercio: int
+ ruta: string
+ texto_alternativo: string
+ orden: int
+ actualizar(datos: array): void
+ comercio(): BelongsTo
CalculoPorcentaje
+ calcular(subtotal: string, valor: string): string
<<Entity>>
PoliticaComercio
+ id_politica: int
+ id_comercio: int
+ tipo: string
+ contenido: string
+ configuracion_envio: array
+ actualizar(datos: array): void
+ comercio(): BelongsTo
<<Entity>>
BannerComercio
+ id_banner: int
+ id_comercio: int
+ ruta: string
+ texto_alternativo: string
+ orden: int
+ activo: bool
+ actualizar(datos: array): void
+ comercio(): BelongsTo
CalculoMontoFijo
+ calcular(subtotal: string, valor: string): string
<<Entity>>
ContactoComercio
+ id_contacto: int
+ id_comercio: int
+ tipo: string
+ valor: string
+ actualizar(datos: array): void
+ comercio(): BelongsTo
<<Entity>>
HorarioComercio
+ id_horario: int
+ id_comercio: int
+ dia_semana: int
+ hora_apertura: ?string
+ hora_cierre: ?string
+ cerrado: bool
+ actualizar(datos: array): void
+ comercio(): BelongsTo
<<Entity>>
SolicitudComercio
+ id_solicitud: int
+ id_comercio: int
+ datos_encargado: array
+ estado: string
+ requiere_documentos: bool
+ motivo_rechazo ?string
+ fecha_reenvio: ?string
+ estaPendiente(): bool
+ aprobar(): void
+ rechazar(motivo: string): void
+ comercio(): BelongsTo

4.1 Resumen de clases

Clase	Capa	Atributos principales	Métodos principales (firma)	Relaciones y navegabilidad
SolicitudesController	Presentación	- comercios: ComerciosService	+ registrar(request: Request): JsonResponse; + aprobar(idSolicitud: int, request: Request): JsonResponse; + rechazar(idSolicitud: int, request: Request): JsonResponse	Depende de ComerciosService y SolicitudComercioPolicy.
ComerciosController	Presentación	- comercios: ComerciosService	+ consultar(idComercio: int): JsonResponse; + suspender(idComercio: int, request: Request): JsonResponse; + reactivar(idComercio: int, request: Request): JsonResponse	Depende de ComerciosService y ComercioPolicy.
MicrositiosController	Presentación	- comercios: ComerciosService	+ consultar(idComercio: int): JsonResponse; + guardarLogotipo(idComercio: int, request: Request): JsonResponse; + gestionarGaleria(idComercio: int, request: Request): JsonResponse	Delega la gestión del micrositio en ComerciosService.
ConfiguracionComercialController	Presentación	- comercios: ComerciosService	+ guardarComision(idComercio: int, request: Request): JsonResponse; + gestionarMembresia(idComercio: int, request: Request): JsonResponse; + gestionarPublicidad(idCo	Depende de ComerciosService y ComercioPolicy.

			mercio: int, request: Request): JsonResponse	
ConsultasComerciosController	Presentación	- comercios: ComerciosServiceInterface	+ publico(idComercio: int): JsonResponse; + estado(idComercio: int): JsonResponse; + comisionSubpedido(idSubpedido: int): JsonResponse	Depende de ComerciosServiceInterface.
ComerciosServiceInterface	Aplicación	Sin atributos.	+ obtenerDatosComercioPublico(idComercio: int): array; + obtenerComercioActivo(idComercio: int): array; + obtenerComisionSubpedido(idSubpedido: int): array	Es implementada por ComerciosService y utilizada por los consumidores de M2.
ComerciosService	Aplicación	Dependencias de identidad, auditoría y pedidos, detalladas en 4.2.	+ aprobarSolicitud(idSolicitud: int): Comercio; + suspenderComercio(idComercio: int, motivo: string): SuspensionComercio; + registrarComisionDespacho(idSubpedido: int, idEvento: string): void	Implementa ComerciosServiceInterface; utiliza las entidades, CalculoComision y los eventos de M2.
RegistrarComision	Aplicación	- comercios: ComerciosService	+ handle(evento: SubpedidoDespachado): void	Recibe el evento de despacho y delega en ComerciosService.
Comercio	Dominio	+ id_comercio: int; + nombre_comercial: string; + estado: string	+ activar(): void; + suspender(): void; + estaActivo(): bool	Tiene 0..1 Micrositio y 0..* registros de las demás entidades del comercio.

				Navegación bidireccional.
SolicitudComercio	Dominio	+ id_solicitud: int; + id_comercio: int; + estado: string	+ estaPendiente(): bool; + aprobar(): void; + rechazar(motivo: string): void	Pertenece a 1 Comercio; un Comercio tiene 0..* solicitudes.
DocumentoComercio	Dominio	+ id_documento_comercio: int; + fecha_vencimiento: ?Carbon; + resultado: string	+ esValido(fecha: Carbon): bool; + evaluar(resultado: string, observacion: ?string): void	Pertenece a 1 Comercio; un Comercio tiene 0..* documentos.
SuspensionComercio	Dominio	+ id_suspension: int; + motivo: string; + fecha_suspension: Carbon	+ reactivar(fecha: Carbon): void; + comercio(): BelongsTo	Pertenece a 1 Comercio; un Comercio tiene 0..* suspensiones.
Micrositio	Dominio	+ id_micrositio: int; + descripcion: string; + logotipo: ?string	+ cambiarLogotipo(ruta: ?string): void; + comercio(): BelongsTo	Pertenece a 1 Comercio; un Comercio tiene 0..1 Micrositio.
BannerComercio, ImagenGaleriaComercio, HorarioComercio, ContactoComercio y PoliticaComercio	Dominio	Banner: + ruta: string, + orden: int; Galería: + ruta: string, + texto_alternativo: string; Horario: + dia_semana: int, + cerrado: bool; Contacto: + tipo: string, + valor: string; Política: + tipo:	En todas: + actualizar(datos: array): void; + comercio(): BelongsTo	Cada registro pertenece a 1 Comercio; un Comercio tiene 0..* registros de cada clase. Navegación bidireccional.

		string, + contenido: string		
ComisionComercio	Dominio	+ tipo: string; + valor: string; + vigente_desde: Carbon; + moneda: decimal; + vigentePara: strign;	+ comercio(): BelongsTo; + membresia(): BelongsTo	Pertenece a 1 Comercio y puede vincularse con 0..1 MembresiaComercio.
ComisionSubpedido	Dominio	+ id_subpedido: int; + subtotal_base: string; + monto: string; + id_comision_subpedido: int; + moneda: decimal; + id_evento: int;	+ comercio(): BelongsTo + existePorSubpedido()	Pertenece a 1 Comercio; un Comercio tiene 0..* comisiones calculadas. Referencia lógica al subpedido de M5.
MembresiaComercio	Dominio	+ nombre: string; + comision_preferencial: ?string; + activa: bool	+ estaVigente(fecha: Carbon): bool; + comercio(): BelongsTo	Pertenece a 1 Comercio; puede relacionarse con 0..* configuraciones de comisión.
PublicidadComercio	Dominio	+ ubicacion: string; + inicio: Carbon; + fin: Carbon	+ estaVigente(fecha: Carbon): bool; + comercio(): BelongsTo	Pertenece a 1 Comercio; un Comercio tiene 0..* publicidades.
CalculoComision	Dominio	Sin atributos.	+ calcular(subtotal: string, valor: string): string	Interfaz utilizada por ComerciosService e implementada

				por las dos estrategias.
CalculoPorcentaje	Dominio	Sin atributos propios.	+ calcular(subtotal: string, valor: string): string	Realiza CalculoComision.
CalculoMontoFijo	Dominio	Sin atributos propios.	+ calcular(subtotal: string, valor: string): string	Realiza CalculoComision.
ComercioAprobado	Dominio	+ id_evento: string; + id_comercio: int; + nombre_comercial: string	+ __construct(id_evento: string, fecha_hora: Carbon, id_comercio: int, estado: string, nombre_comercial: string)	ComerciosService lo crea y publica. Consumidores: M1, M3 y M4.
ComercioSuspendido	Dominio	+ id_evento: string; + id_comercio: int; + motivo: string	+ __construct(id_evento: string, fecha_hora: Carbon, id_comercio: int, estado: string, motivo: string)	ComerciosService lo crea y publica. Consumidores: M1, M3, M4 y M5.
ComercioReactivado	Dominio	+ id_evento: string; + id_comercio: int; + nombre_comercial: string	+ __construct(id_evento: string, fecha_hora: Carbon, id_comercio: int, estado: string, nombre_comercial: string)	ComerciosService lo crea y publica. Consumidores: M1, M3, M4 y M5.
SolicitudComercioPolicy	Infraestructura	Sin atributos propios.	+ revisar(usuario: Usuario, solicitud: SolicitudComercio): bool; + aprobar(usuario: Usuario, solicitud: SolicitudComercio): bool; + rechazar(usuario: Usuario, solicitud: SolicitudComercio): bool	Es utilizada por SolicitudesController; depende de SolicitudComercio y usa la verificación de roles del núcleo.

ComercioPolicy	Infraestructura	Sin atributos propios.	+ actualizar(usuario: Usuario, comercio: Comercio): bool; + suspender(usuario: Usuario, comercio: Comercio): bool; + reactivar(usuario: Usuario, comercio: Comercio): bool	Es utilizada por los controladores administrativos; depende de Comercio y usa la verificación de roles del núcleo.

4.2 Clases del núcleo y de otros módulos que se usan

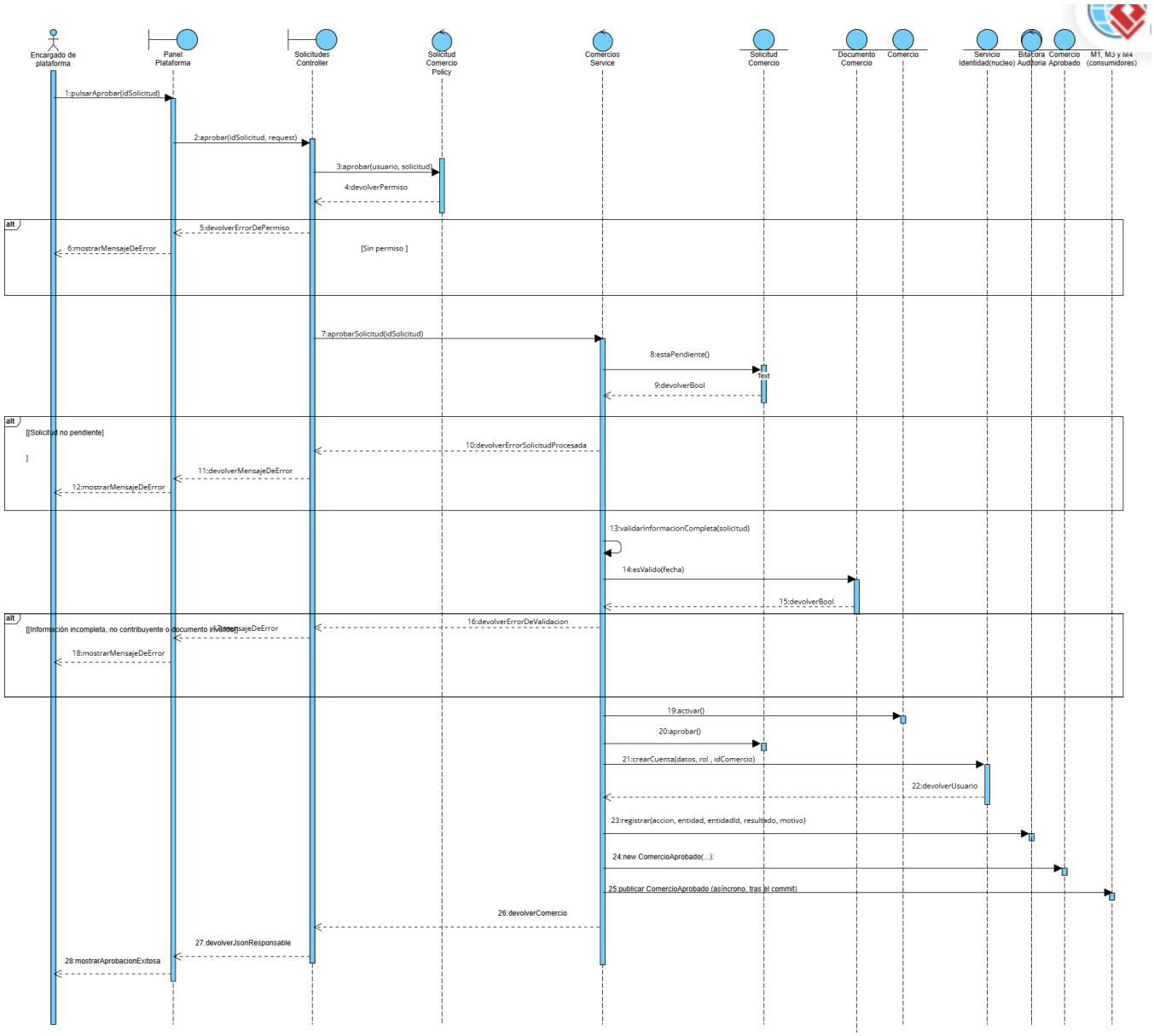
Clase o interfaz externa	Origen	Para qué la usa el módulo
PerteneceAComercio	Núcleo — trait	Aplicar el aislamiento por comercio a las entidades de M2 que contienen id_comercio.
ServicioIdentidad	Núcleo	Crear la cuenta del encargado, asociada al comercio, cuando se aprueba su solicitud.
BitacoraAuditoria	Núcleo	Registrar las aprobaciones y suspensiones de comercios con el usuario responsable y el resultado.
VerificaRol	Núcleo — trait	Permitir que las Policies comprueben el rol del usuario antes de autorizar una acción.
Usuario	Núcleo	Representar al usuario que reciben como parámetro los métodos de las Policies.
PedidosServiceInterface	M5 — interfaz	Consultar obtenerSubpedido para obtener el subtotal utilizado en el cálculo de la comisión.
SubpedidoDespachado	M5 — evento	Disparar RegistrarComision para calcular y guardar la comisión del subpedido despachado.

5. Diagramas de interacción actualizados

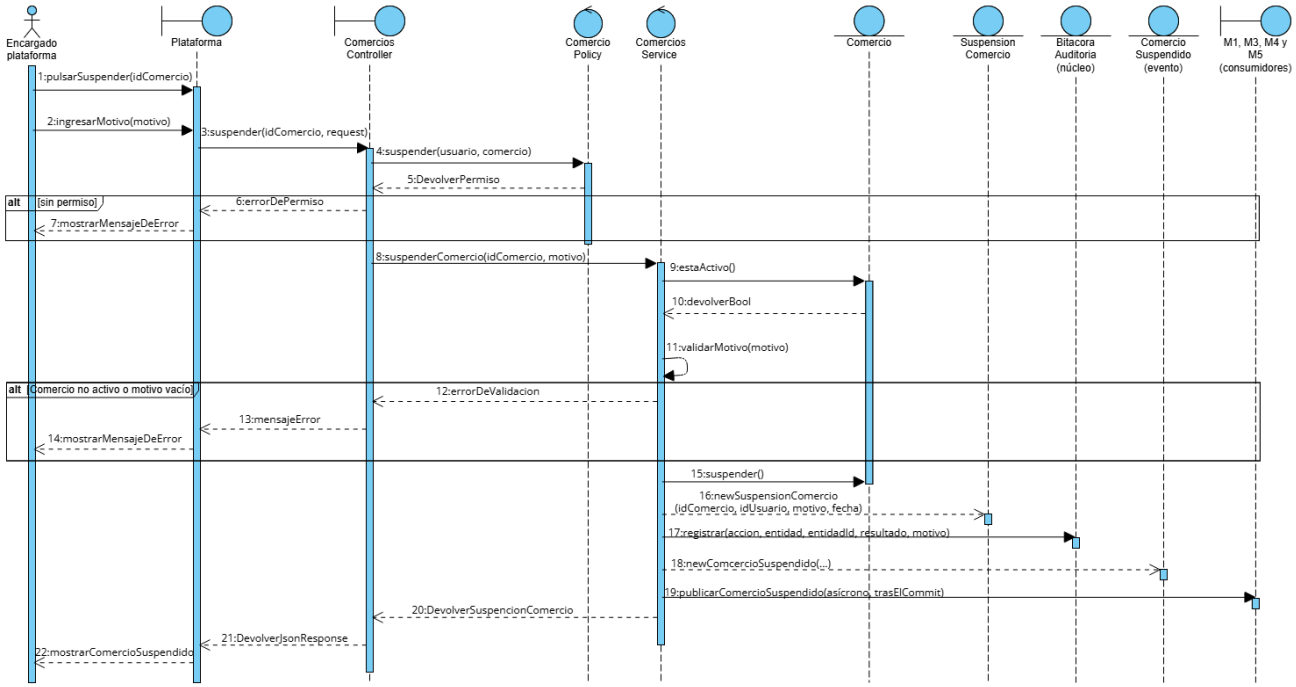
Diagramas de secuencia de los casos de uso del PC2 actualizados con las clases de diseño, incluyendo las llamadas a otros módulos.

Diagrama	Caso de uso	Módulos externos llamados	Operación o evento del contrato usado
Ds-01	CU-0015 Aprobar solicitud de comercio	Núcleo; M1, M3 y M4 (consumidores del evento)	Núcleo: ServicioIdentidad::crearCuenta y BitacoraAuditoria::registrar (sección 5 del colectivo). M2 publica ComercioAprobado v1 (contrato M2 v1.2.1).
Ds-02	CU-0017 Suspender comercio	Núcleo; M1, M3, M4 y M5 (consumidores del evento)	Núcleo: BitacoraAuditoria::registrar. M2 publica ComercioSuspendido v1 (contrato M2 v1.2.1).
Ds-03	Registrar comisión del subpedido (RN-010, paso 9 del recorrido transversal)	M5	M5 publica SubpedidoDespachado v1.0.0 y M2 lo consume. M2 llama a obtenerSubpedido de PedidosServiceInterface (contrato M5 v1.1.0).

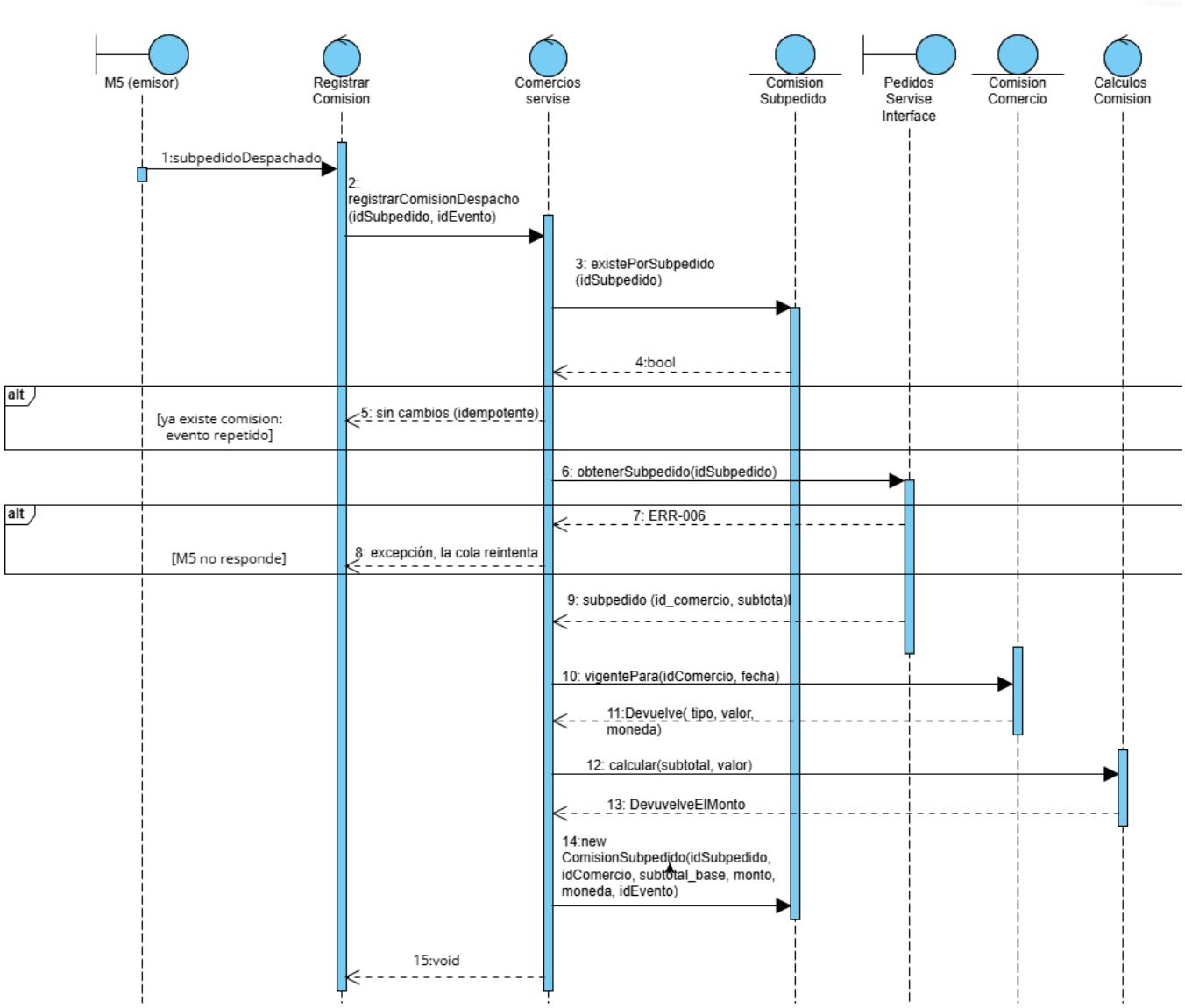
Ds-01:


Encargado de plataforma
Panel Plataforma
Solicitudes Controller
Solicitud Comercio Policy
Comercios Service
Solicitud Comercio
Documento Comercio
Comercio
Servicio Identidad(nucleo)
Bitacora Auditoria
Comercio Aprobado
MT, M3 y M4 (consumidores)
1:pulsarAprobar(idSolicitud)
2:aprobar(idSolicitud, request)
3:aprobar(usuario, solicitud)
4:devolverPermiso
alt
5:devolverErrorDePermiso
6:mostrarMensajeDeError
[Sin permiso]
7:aprobarSolicitud(idSolicitud)
8:estaPendiente()
9:devolverBool
alt
[[Solicitud no pendiente]
]
10:devolverErrorSolicitudProcesada
11:devolverMensajeDeError
12:mostrarMensajeDeError
13:validarInformacionCompleta(solicitud)
14:esValido(fecha)
15:devolverBool
alt
[[Información incompleta, no contribuyente o documento inválido]]
16:devolverErrorDeValidacion
17:mensajeDeError
18:mostrarMensajeDeError
19:activar()
20:aprobar()
21:crearCuenta(datos, rol , idComercio)
22:devolverUsuario
23:registrar(accion, entidad, entidadId, resultado, motivo)
24:new ComercioAprobado(...):
25:publicar ComercioAprobado (asíncrono, tras el commit)
26:devolverComercio
27:devolverJsonResponsable
28:mostrarAprobacionExitosa

Ds-02:


Encargado plataforma
Plataforma
Comercios Controller
Comercio Policy
Comercios Service
Comercio
Suspension Comercio
Bitacora Auditoria (núcleo)
Comercio Suspendido (evento)
M1, M3, M4 y M5 (consumidores)
1:pulsarSuspender(idComercio)
2:ingresarMotivo(motivo)
3:suspender(idComercio, request)
4:suspender(usuario, comercio)
5:DevolverPermiso
alt
[sin permiso]
6:errorDePermiso
7:mostrarMensajeDeError
8:suspenderComercio(idComercio, motivo)
9:estaActivo()
10:devolverBool
11:validarMotivo(motivo)
alt
[Comercio no activo o motivo vacío]
12:errorDeValidacion
13:mensajeError
14:mostrarMensajeDeError
15:suspender()
16:newSuspensionComercio (idComercio, idUsuario, motivo, fecha)
17:registrar(accion, entidad, entidadId, resultado, motivo)
18:newComcercioSuspendido(...)
19:publicarComercioSuspendido(asíncrono, trasElCommit)
20:DevolverSuspencionComercio
21:DevolverJsonResponse
22:mostrarComercioSuspendido

Ds-03:


M5 (emisor)
Registrar Comision
Comercios servise
Comision Subpedido
Pedidos Servise Interface
Comision Comercio
Calculos Comision
1:subpedidoDespachado
2: registrarComisionDespacho (idSubpedido, idEvento)
3: existePorSubpedido (idSubpedido)
4:bool
alt
[ya existe comision: evento repetido]
5: sin cambios (idempotente)
6: obtenerSubpedido(idSubpedido)
alt
7: ERR-006
[M5 no responde]
8: excepción, la cola reintenta
9: subpedido (id_comercio, subtota)
10: vigentePara(idComercio, fecha)
11:Devuelve(tipo, valor, moneda)
12: calcular(subtotal, valor)
13: DevuvelveElMonto
14:new ComisionSubpedido(idSubpedido, idComercio, subtotal_base, monto, moneda, idEvento)
15:void

6. Modelo de datos del módulo

Modelo en tercera forma normal, implementado como migraciones de Laravel, con restricciones de unicidad y verificación que implementan las reglas de negocio. Se entrega además el SQL resultante.

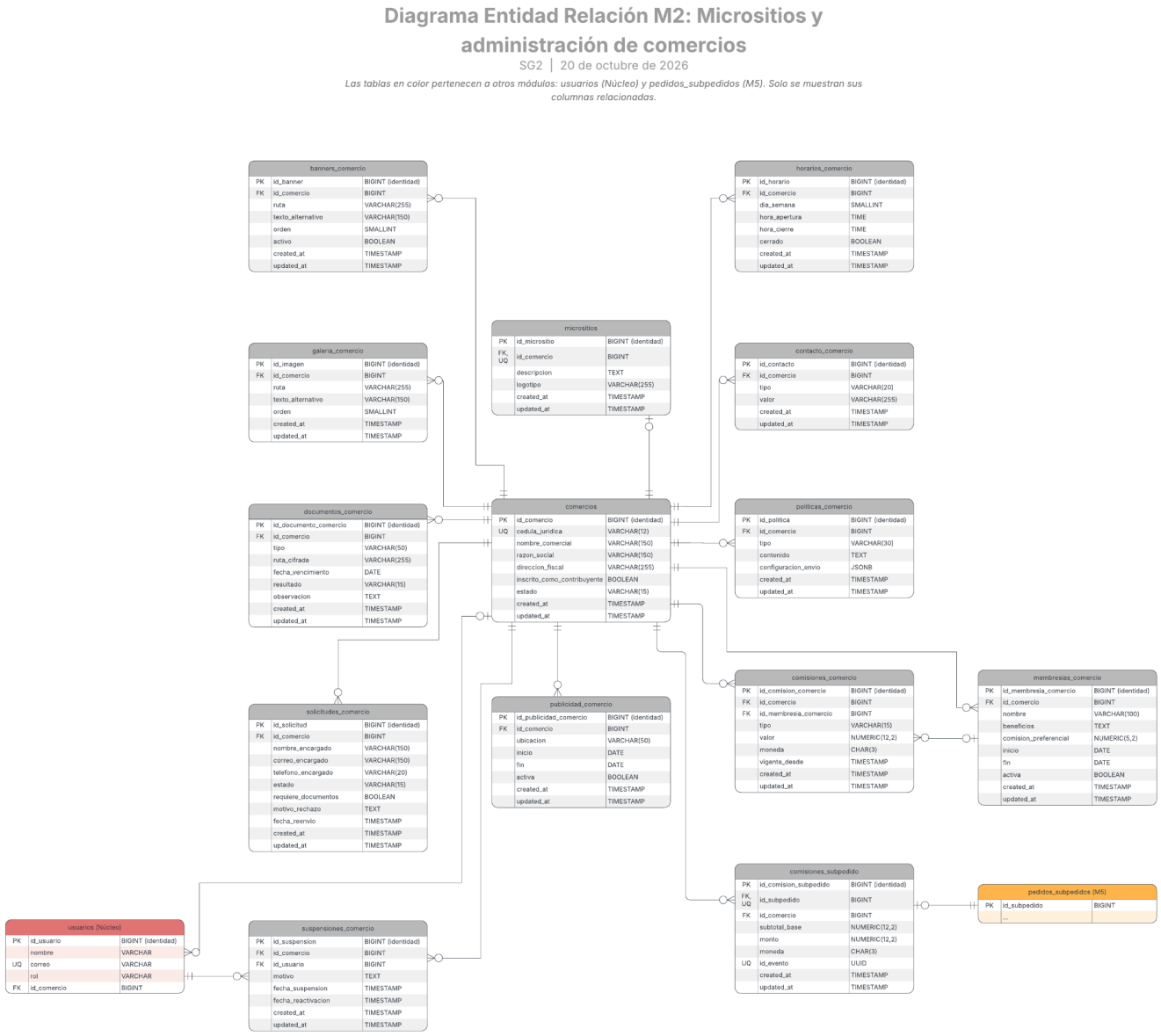
6.1 Tablas del módulo

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
comercios	id_comercio	Ninguna	Es la tabla principal del módulo dos que almacena los datos base y el estado operativo del comercio dentro de Pacíficos Online. Respalda la consulta y actualización de la información empresarial y su ciclo de incorporación, aprobación, rechazo, suspensión y reactivación de dicho comercio (CU-0001, CU-0002, CU-0008, CU-0012 a CU-0018; RN-001, RN-004, RN-005, RN-011, RN-016, RN-017)
micrositios	id_micrositio	id_comercio	Es la tabla hija de comercios que representa el espacio público y personalizado asociado a cada comercio del entorno. Funciona de base para organizar la información de presentación y los elementos administrados desde el módulo dos como la información empresarial y los componentes visuales del micrositio (CU-0002, CU-0007)
comisiones_comercio	id_comision_comercio	id_comercio, id_membresia_comercio	Es un esquema de la comisión que la plataforma se encarga de asignarle a comercio, como tipo (porcentaje o monto fijo),

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
			el valor, la fecha de entrada en vigencia y la membresía de origen si es que la comisión es de tipo preferencial. Además conserva el historial de los esquemas esto para no llegar a alterar retrospectivamente comisiones que fueron anteriormente registradas. Si existe una comisión que provenga de una membresía se asocia con la segunda clave compartida (RN-006, RN-007, RN-008, RN-017)
comisiones_subpedido	id_comision_subpedido	id_subpedido, id_comercio	Se encarga de registrar la comisión calculada para cada subpedido que se despacha. Comisión es calculada sobre el subtotal del subpedido al procesar el despacho (RN-010).
membresias_comercio	id_membresia_comercio	id_comercio	Guarda las membresías activas asociadas a cada comercio (CU-0020, RF-020, RN-008, RN-017).
publicidad_comercio	id_publicidad_comercio	id_comercio	Guarda las pautas de publicidad asignadas a un comercio para gestionar la promoción y su posición destacada (CU-0021, RF-021).
suspensiones_comercio	id_suspension	Id_comercio (va hacia comercios) /id_usuario (Nucleo, quien ejecutó la suspensión).	Este es el historial de suspensiones, especificado de un comercio, tiene motivo obligatorio, fecha de suspensión y la fecha de

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
			reactivación (RN-004, RN-005, CU-0017, CU-0018).
solicitudes_comercio	id_solicitud	id_comercio	Registra el trámite de solicitud de incorporación de un comercio, permite conservar información relacionada a su revisión y el estado de la solicitud (CU-0012 a CU-0016).
contacto_comercio	id_contacto	id_comercio	Almacena la información de contacto que está asociada a cada comercio, la cual incluye: teléfonos, correos electrónicos y redes sociales. También, da soporte a la consulta y actualización de sus redes sociales. CU-0001, CU-0002 y CU-0007.
politicas_comercio	id_politica	id_comercio	Almacena las políticas asociadas a cada comercio para que formen parte de la información administrada y publicada en su micrositio. CU-0022 y RN-002
documentos_comercio	id_documento_comercio	id_comercio	Almacena los documentos asociados a cada comercio y la información necesaria para registrar su evaluación durante el proceso de incorporación y aprobación. CU-0022 y RN-002
banners_comercio	id_banner	id_comercio (llave foránea interna hacia comercios)	Almacena los banners del comercio para permitir su registro, cambio o eliminación y conservar los cambios para consultas posteriores

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
			(CU-0004: Administrar banner; RF-004 del PC2).
horarios_comercio	id_horario	id_comercio (llave foránea interna hacia comercios)	Almacena los horarios de atención del comercio para permitir su registro, modificación o eliminación y conservar los cambios para consultas posteriores (CU-0006: Administrar horarios; RF-006 del PC2).
galeria_comercio	id_imagen	id_comercio	Guarda las imágenes de la galería subidas asociadas a un comercio (CU-0005, RF-005)



6.2 Verificación de tercera forma normal

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
comercios	Cumple	Cumple	Cumple	Atributos del comercio solo dependen de id_comercio
micrositios	Cumple	Cumple	Cumple	Atributos del micrositito dependen de id_micrositio

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
comisiones_comercio	Cumple	Cumple	Cumple	Cada registro representa una comisión vigente. Tipo, valor, moneda y membresía dependen de id_comision_comercio
comisiones_subpedido	Cumple	Cumple	Cumple	Cada registro representa una comisión calculada para el sub pedido. id_subpedido es UNIQUE para evitar duplicidad
membresias_comercio	Ajuste	Cumple	Cumple	atributos beneficios se guarda como text y no como arreglo
publicidad_comercio	Cumple	Cumple	Cumple	atributos dependen únicamente de id_publicidad_comercio
suspensiones_comercio	Cumple	Cumple	Cumple	Cada suspensión es un registro independiente
solicitudes_comercio	Ajuste	Cumple	Cumple	Separar datos de datos_encargado en columnas como nombre, correo y teléfono
contacto_comercio	Cumple	Cumple	Cumple	Cada valor se registra individualmente mediante tipo y valor

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
politicas_comercio	Cumple	Cumple	Cumple	tipo y contenido son valores atómicos
documentos_comercio	Cumple	Cumple	Cumple	Cada documento se almacena en un registro separado y sus atributos dependen únicamente de id_documento_comercio
banners_comercio	Cumple	Cumple	Cumple	Cada banner se registra como registro independiente y depende solo de id_banner
horarios_comercio	Cumple	Cumple	Cumple	Cada registro representa un horario de un dia en especifico
galeria_comercio	Cumple	Cumple	Cumple	Cada imagen se registra individualmente, sus atributos dependen solo de id_imagen

6.3 Restricciones que implementan reglas de negocio

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
RN-001: Solo se aprueba un comercio inscrito como contribuyente	comercios	not null	inscrito_como_contribuyente debe tener un valor booleano y no null	inscrito_como_contribuyente_not_null
RN-002: Los documentos requeridos deben estar completos, vigentes y válidos	documentos_comercio	not null, check	ruta_cifrada no puede ser null. resultado solo puede tener valores	chk_documentos_resultado

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
			pendiente, valido o invalido	
RN-005: Toda suspensión requiere un motivo	suspensiones_comercio	not null, check	motivo no puede ser null	chk_suspensiones_motivo
RN-006: La comisión debe estar dentro del rango permitido	comisiones_comercio	check	valor >= 0 AND (tipo <> 'porcentaje' OR valor <= 100)	chk_comisiones_valor
RN-007: La comisión solo puede configurarse como porcentaje o monto fijo	comisiones_comercio	check	tipo IN ('porcentaje', 'monto_fijo')	chk_comisiones_tipo
RN-008: Una comisión preferencial puede provenir de una membresía	comisiones_comercio y membresias_comercio	fk, check	id_membresia_comercio es un fk interna nullable hacia membresias_comercio	fk_comision_membresia, chk_membresia_comision_preferencial
RN-010: La comisión se calcula sobre el subtotal del subpedido	comisiones_subpedido	not null, check, unique	subtotal_base no puede ser nulo y debe cumplir subtotal_base >= 0	chk_comision_subpedido_subtotal, chk_comision_subpedido_monto, uq_comision_subpedido
RN-011: Una misma solicitud no puede aprobarse dos veces	solicitudes_comercio	unique parcial	Solo puede existir una solicitud en estado pendiente por comercio	uq_solicitud_pendiente_comercio
RN-016: Todo rechazo requiere un motivo	solicitudes_comercio	check	motivo_rechazo debe existir y no estar vacío	chk_solicitud_motivo_rechazo
RN-015 / RN-017: Se conserva el historial de configuraciones de comisión y estas se mantienen al reactivar el comercio	comisiones_comercio	unique	La combinación id_comercio, vigente_desde debe ser única	uq_comision_comercio_vigencia
El comercio solo puede encontrarse en uno de los cuatro estados	comercios	check	estado IN ('Pendiente', 'Activo', 'Rechazado', 'Suspendido')	chk_comercios_estado

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
Cédula jurídica debe de respetar formato y ser única	comercios	unique, check	cumplir el formato ^[0-9]-[0-9]{3}-[0-9]{6}$	uq_comercios_cedula_juridica, chk_comercios_cedula_formato

6.4 Migraciones

Archivo de migración	Tabla	Descripción
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	comercios	Crea la tabla principal de M2 con la información empresarial y estado del comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	micrositios	Crea el micrositio de cada comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	comisiones_comercio	Crea el historial de configuraciones de comisión de un comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	comisiones_subpedido	Crea el registro de comisión calculada por cada subpedido que se despacha
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	membresias_comercio	Crea las membresías vinculadas a cada comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	publicidad_comercio	Crea pautas publicitarias que son asignadas a cada comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	suspensiones_comercio	Crea un historial de los comercios que se encuentran en estado de suspensión
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	solicitudes_comercio	Crea las solicitudes de incorporación de cada comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	contacto_comercio	Crea los medios de contacto y redes sociales del comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	politicas_comercio	Crea las políticas relacionadas a cada comercio

Archivo de migración	Tabla	Descripción
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	documentos_comercio	Crea los documentos que se usan para verificar la incorporación del comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	banners_comercio	Crea los banners del comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	horarios_comercio	Crea los horarios de atención del comercio
database/migrations/M2_Comercios/2026_10_21_100000_create_comercios_table.php	galeria_comercio	Crea las imágenes que forman parte de la galería de cada comercio

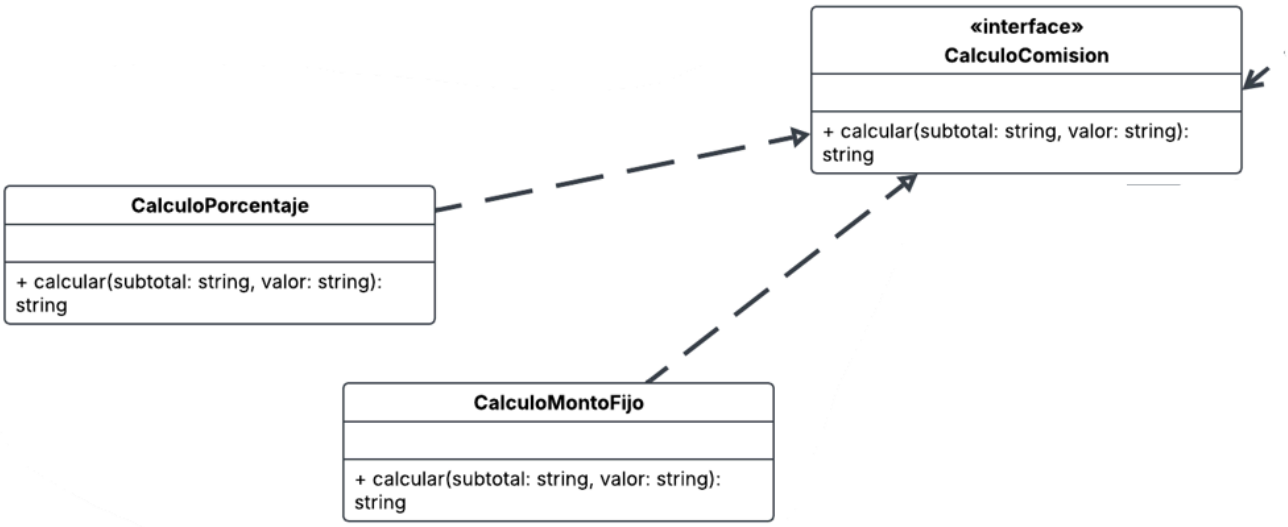
6.5 SQL resultante

La generación del SQL se realiza mediante *php artisan migrate --pretend*. Esta quedó almacenada en db/M2_Comercios/sql_generated.txt.

7. Patrones de diseño aplicados (mínimo tres)

7.1 Strategy

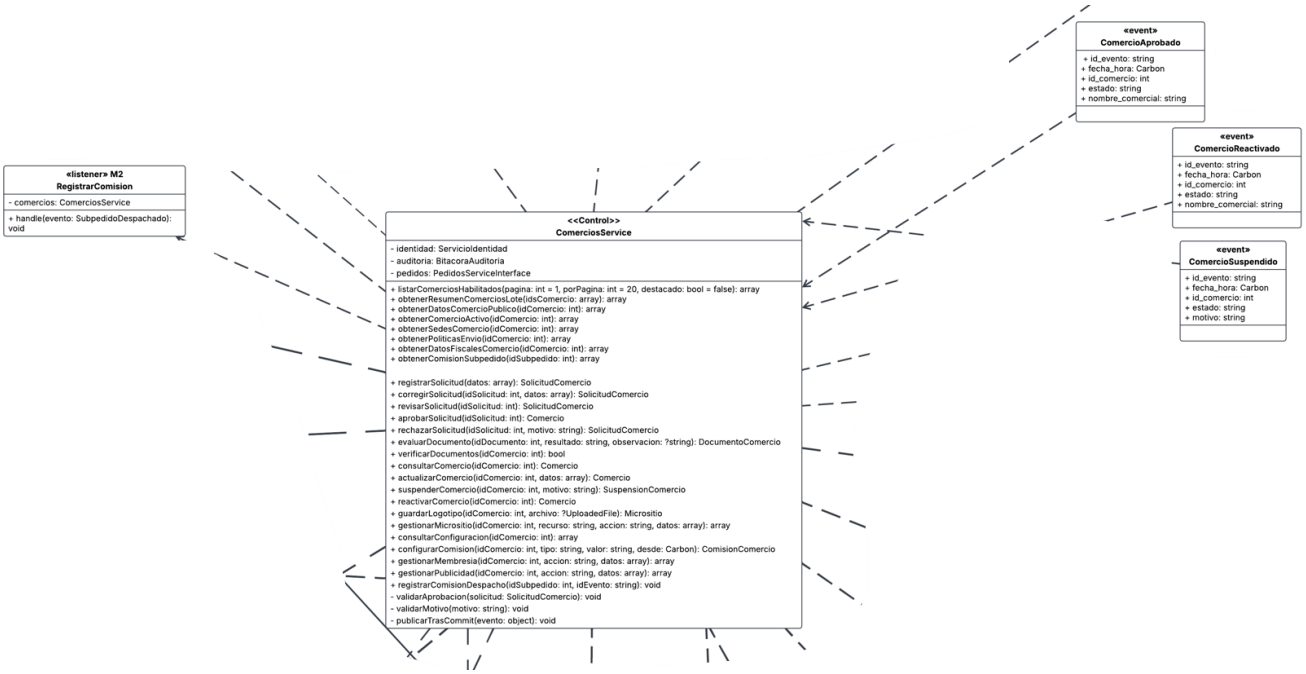
Categoría	Comportamiento
Problema del módulo que resuelve	M2 debe calcular comisiones utilizando más de una modalidad, concretamente porcentaje o monto fijo. Si toda la lógica de cálculo se implementara mediante condicionales dentro de ComerciosService, el servicio quedaría acoplado a cada forma de cálculo y sería más difícil incorporar nuevas modalidades.
Dónde se aplica	CalculoComision, CalculoPorcentaje, CalculoMontoFijo y ComerciosService
Justificación	Strategy encapsula cada forma de cálculo de comisión detrás de la interfaz CalculoComision. ComerciosService puede seleccionar la estrategia correspondiente según el tipo de comisión configurado sin conocer los detalles de la fórmula. Esto se ajusta a RN-007, que admite comisión por porcentaje o monto fijo, y a RN-010, que establece que la base del cálculo es el subtotal del subpedido.
Alternativa descartada	Implementar un único método de cálculo con condicionales if/switch según el tipo de comisión. Se descarta porque concentra varias reglas de cálculo en una sola clase y aumenta el acoplamiento al agregar nuevas modalidades.



7.2 Observer

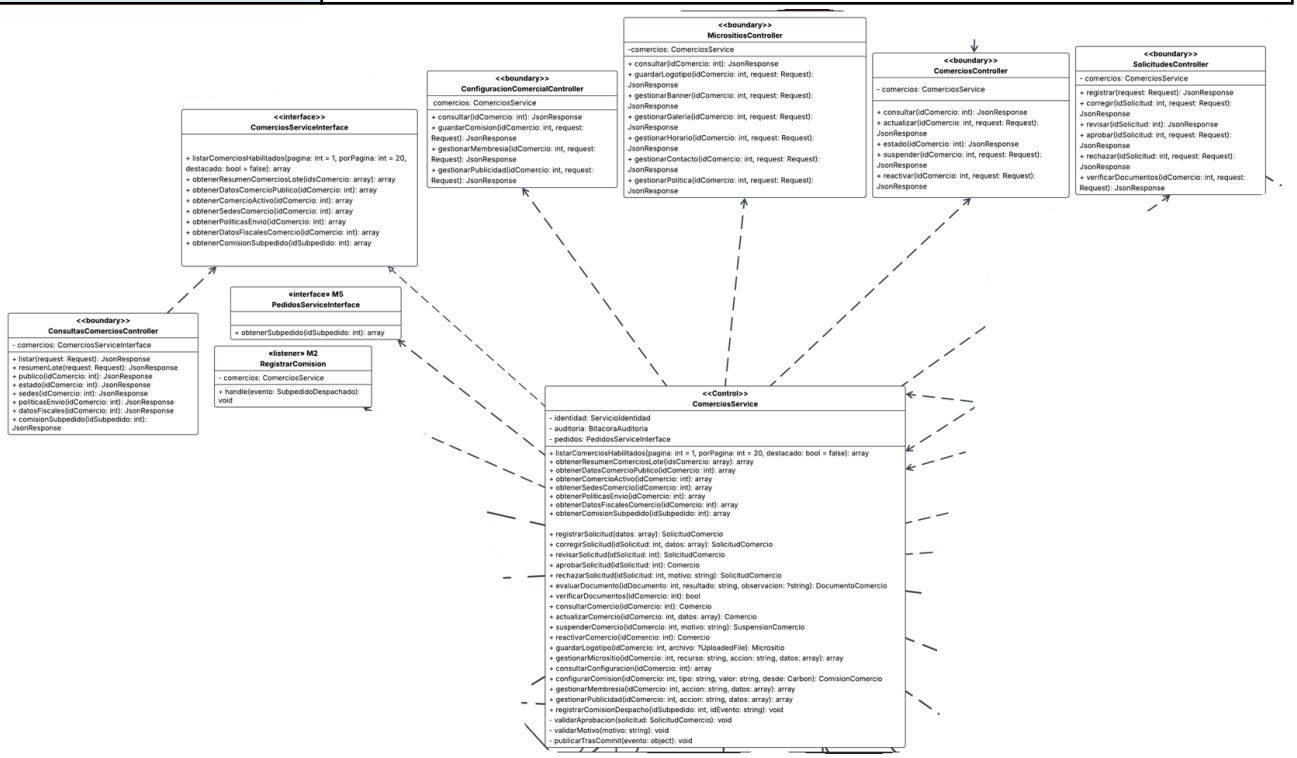
Categoría	Comportamiento
Problema del módulo que resuelve	M2 debe reaccionar a eventos producidos por otros módulos y comunicar cambios relevantes del estado de los comercios sin realizar llamadas

	directas y fuertemente acopladas entre las clases internas de los distintos módulos.
Dónde se aplica	ComercioAprobado, ComercioSuspendido, ComercioReactivado, RegistrarComision, ComerciosService y el evento externo SubpedidoDespachado
Justificación	El patrón Observer permite reaccionar a eventos sin que el productor necesite conocer directamente a todos los consumidores. RegistrarComision funciona como listener del evento SubpedidoDespachado producido por M5 y delega el cálculo y registro de la comisión en ComerciosService. De manera inversa, M2 publica los eventos ComercioAprobado, ComercioSuspendido y ComercioReactivado, que pueden ser procesados por otros módulos. Esto mantiene un bajo acoplamiento y respeta el mecanismo de integración asíncrona definido para Pacíficos Online.
Alternativa descartada	Hacer llamadas directas desde M5 hacia las clases internas de M2 o desde ComerciosService hacía cada módulo consumidor al aprobar, suspender o reactivar un comercio. Se descarta porque aumentaría el acoplamiento entre módulos y contradiría el uso de eventos establecido por la arquitectura del proyecto.



7.3 Facade

Categoría	Estructural
Problema del módulo que resuelve	M2 contiene numerosas entidades, servicios externos, reglas de negocio y operaciones relacionadas con comercios, micrositios, solicitudes, comisiones, suspensiones y configuración comercial. Si los controladores tuvieran que interactuar directamente con todas estas clases, quedarían fuertemente acoplados a la estructura interna del módulo.
Dónde se aplica	Principalmente en ComerciosService y ComerciosServiceInterface. Es utilizado por ConsultasComerciosController, ConfiguracionComercialController, MicrositiosController, ComerciosController, SolicitudesController y RegistrarComision
Justificación	ComerciosService proporciona un punto central de acceso a la lógica del módulo. Los controladores y listeners delegan en este servicio operaciones como consultar comercios, gestionar micrositios, aprobar o rechazar solicitudes, suspender o reactivar comercios y calcular comisiones. El servicio oculta a sus consumidores la interacción con las entidades de dominio, los servicios del núcleo y los contratos externos. De esta manera, las clases límite trabajan con una interfaz simplificada sin depender directamente de la complejidad interna de M2
Alternativa descartada	Hacer que cada controlador interactúe directamente con entidades como Comercio, SolicitudComercio, ComisionComercio, SuspensionComercio, servicios del núcleo y contratos de otros módulos. Se descarta porque aumentaría el acoplamiento, duplicaría lógica de coordinación y dificultaría el mantenimiento.



8. Prototipo de interfaz del módulo, validado

8.1 Pantallas del prototipo

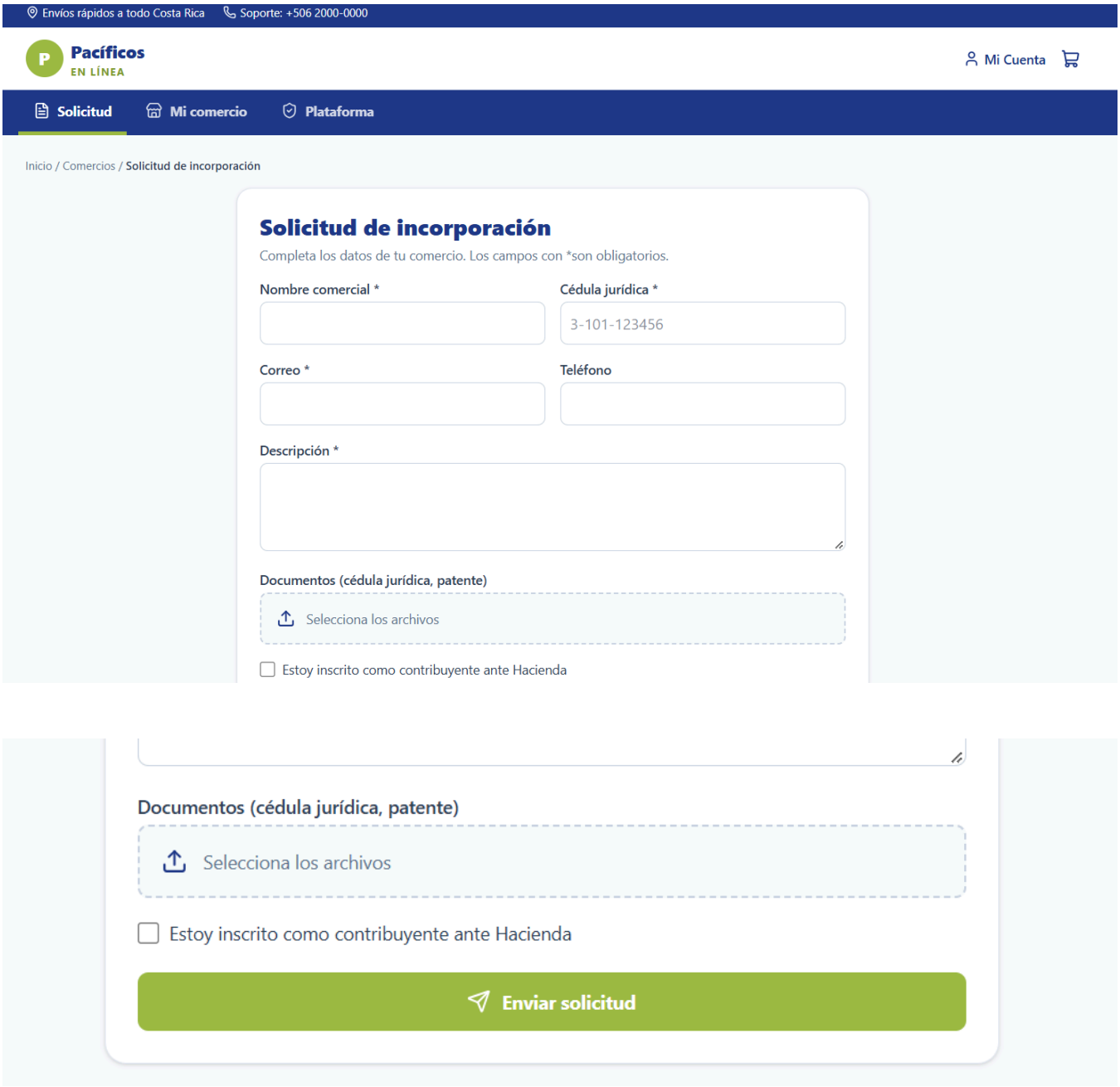
Pantalla	Caso de uso	Actor	Ubicación (repositorio o enlace)
Solicitud de incorporación	CU-0012 Registrar solicitud de incorporación	Encargado del comercio	frontend/prototipos/M2_Comercios/
Panel del comercio: estado y micrositio	CU-0008 Consultar estado del comercio; CU-0002 Actualizar información empresarial; CU-0003 a CU-0007 (logotipo, banner, galería, horarios y redes sociales)	Encargado del comercio	frontend/prototipos/M2_Comercios/
Listado de solicitudes y comercios	CU-0014 Revisar solicitud de incorporación	Encargado de la plataforma	frontend/prototipos/M2_Comercios/
Detalle de la solicitud: aprobar o rechazar	CU-0015 Aprobar solicitud; CU-0016 Rechazar solicitud; CU-0022 Verificar documentos	Encargado de la plataforma	frontend/prototipos/M2_Comercios/
Suspender y reactivar comercio	CU-0017 Suspender comercio; CU-0018 Reactivar comercio	Encargado de la plataforma	frontend/prototipos/M2_Comercios/
Registrar solicitud desde la plataforma	CU-0012 Registrar solicitud de incorporación	Encargado de la plataforma	frontend/prototipos/M2_Comercios/

8.2 Validación

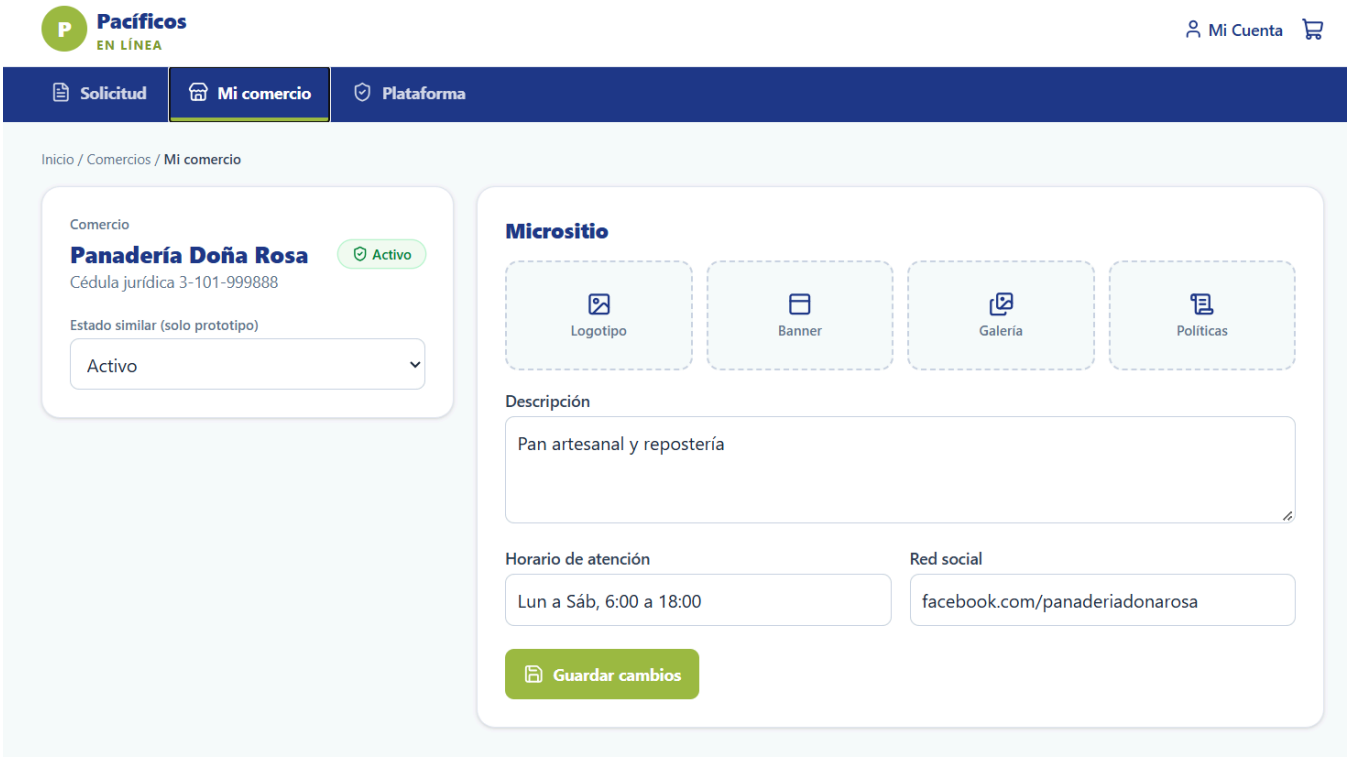
Fecha	21/10/2026
Participantes	Diego Alfaro Alvarado
Método	Realizando un recorrido por las pantallas, comparándolas con los casos de uso y las reglas de negocio del PC2, además con la guía visual planteada por el comite.
Tareas evaluadas	Registrar una solicitud; revisar, aprobar y rechazar solicitudes; suspender y reactivar un comercio; editar el micrositio según el estado del comercio

Hallazgo	Pantalla	Cambio realizado	Estado
No existía forma de reactivar un comercio suspendido (CU-0018)	Suspender y reactivar comercio	Se agregó el botón "Reactivar comercio" para los comercios en estado Suspendido	Aplicado
El panel solo mostraba el estado Activo, por lo que no se podían revisar los avisos de suspensión ni de solicitud pendiente	Panel del comercio: estado y micrositio	Se agregó un selector para simular el estado (Activo, Suspendido, Pendiente) con sus avisos	Aplicado
El Encargado de la plataforma no podía registrar una solicitud, aunque la elicitación indicó que ambos actores pueden iniciarla (CU-0012)	Registrar solicitud desde la plataforma	Se agregó el botón "Registrar solicitud", que abre el mismo formulario	Aplicado
El micrositio no incluía galería ni políticas	Panel del comercio: estado y micrositio	Se agregaron los espacios de galería y políticas	Aplicado
Se podía editar el micrositio de un comercio que no estaba Activo	Panel del comercio: estado y micrositio	Los campos y el botón "Guardar cambios" se deshabilitan si el comercio no está Activo	Aplicado
El diseño no seguía la Guía Visual del Comité (paleta, encabezado, íconos y uso de emojis)	Todas las pantallas	Se aplicó la paleta azul marino y verde, el encabezado común de la plataforma y los íconos de lucide-react, y se eliminaron los emojis	Aplicado

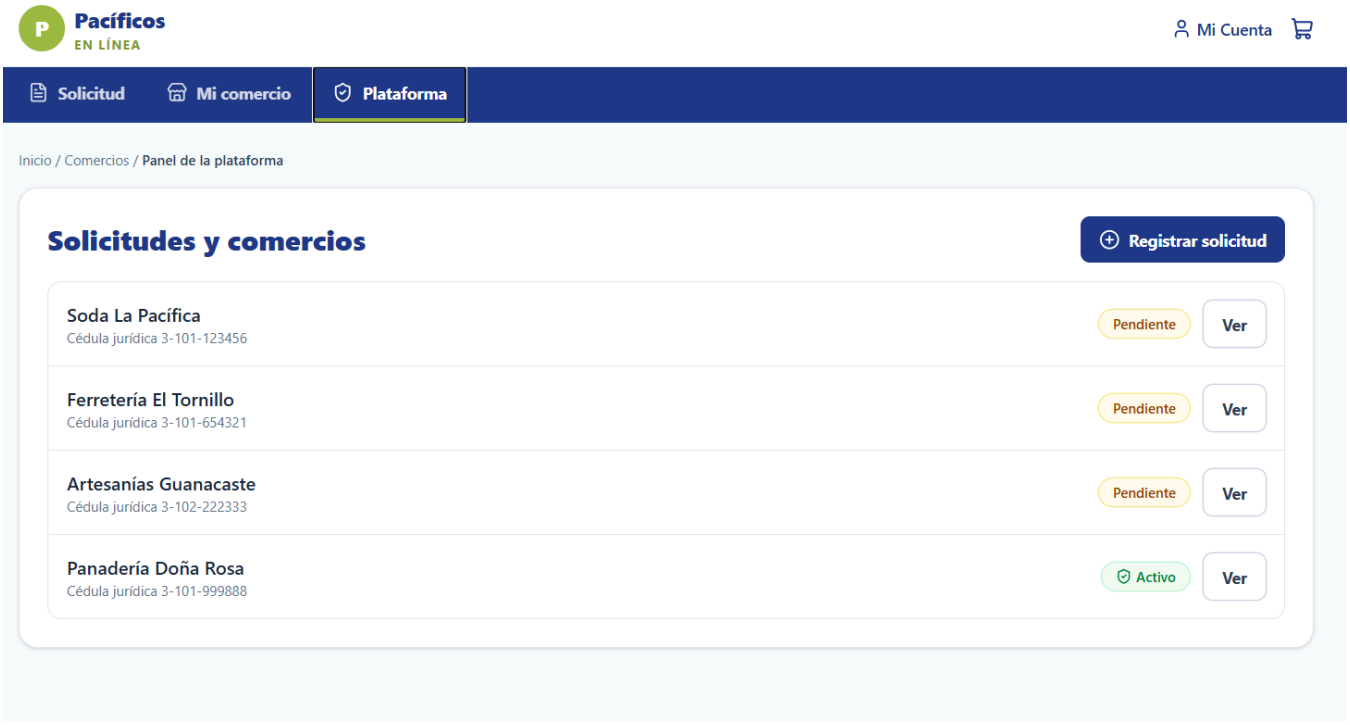
1.Imagen de la pantalla de solicitud para un comercio



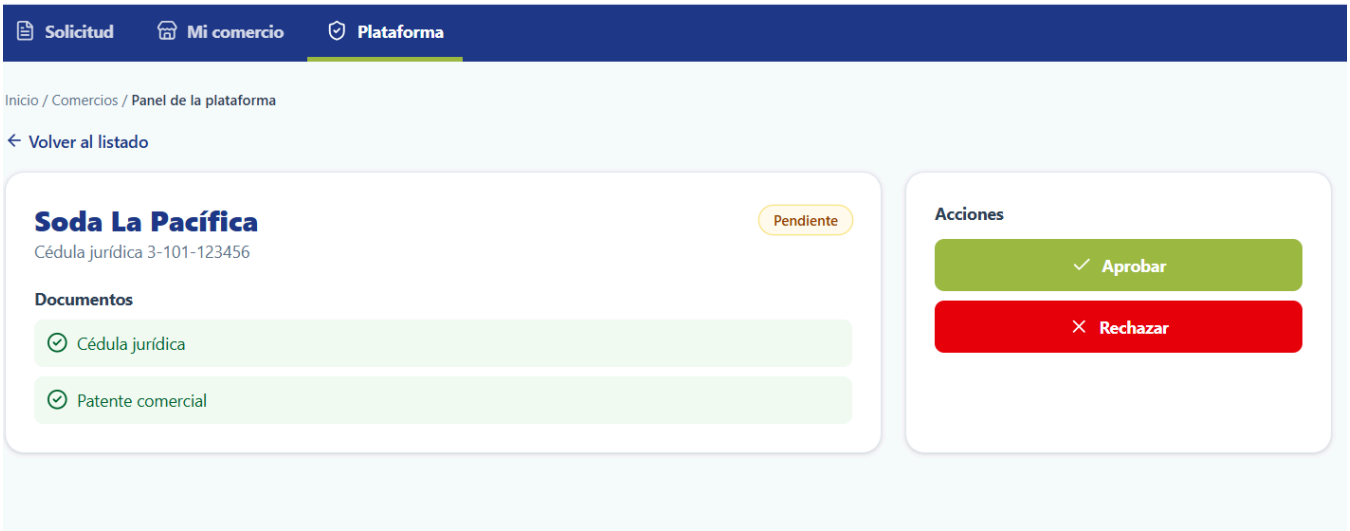
2.Imagen de la pantalla para ingresar a profundidad datos del comercio



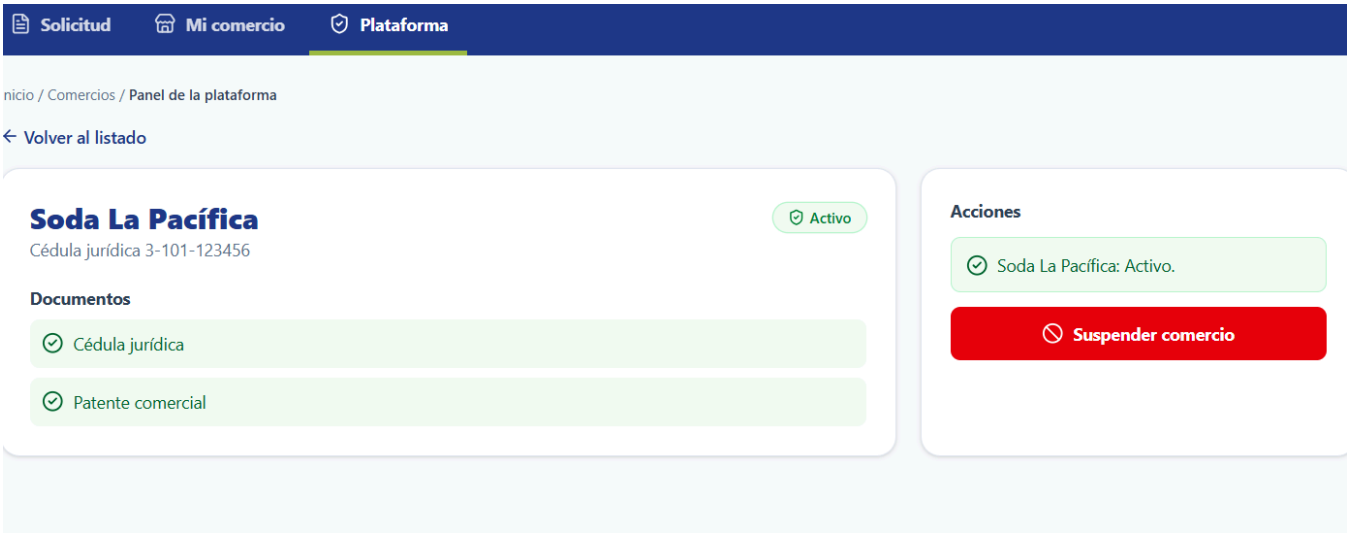
3.Imagen de la pantalla en donde se pueden ver todas las solicitudes de comercio, además de su estado



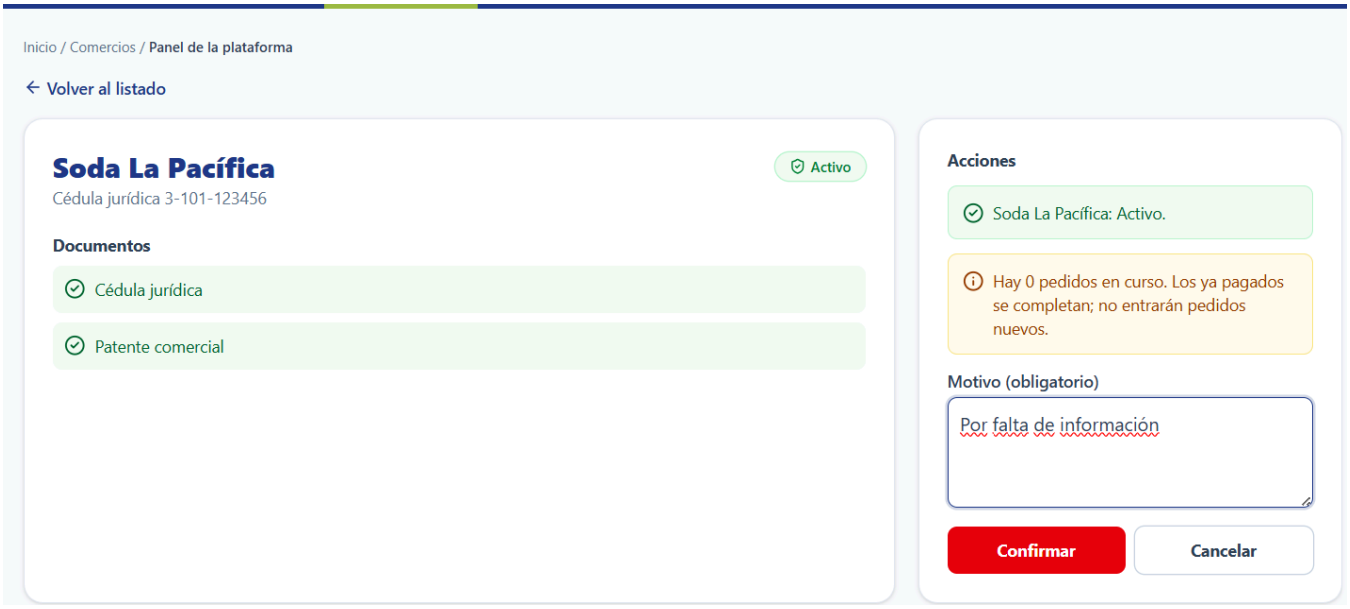
4. Imagen de la pantalla de plataforma, donde se especifica si se va a aprobar un comercio o no.



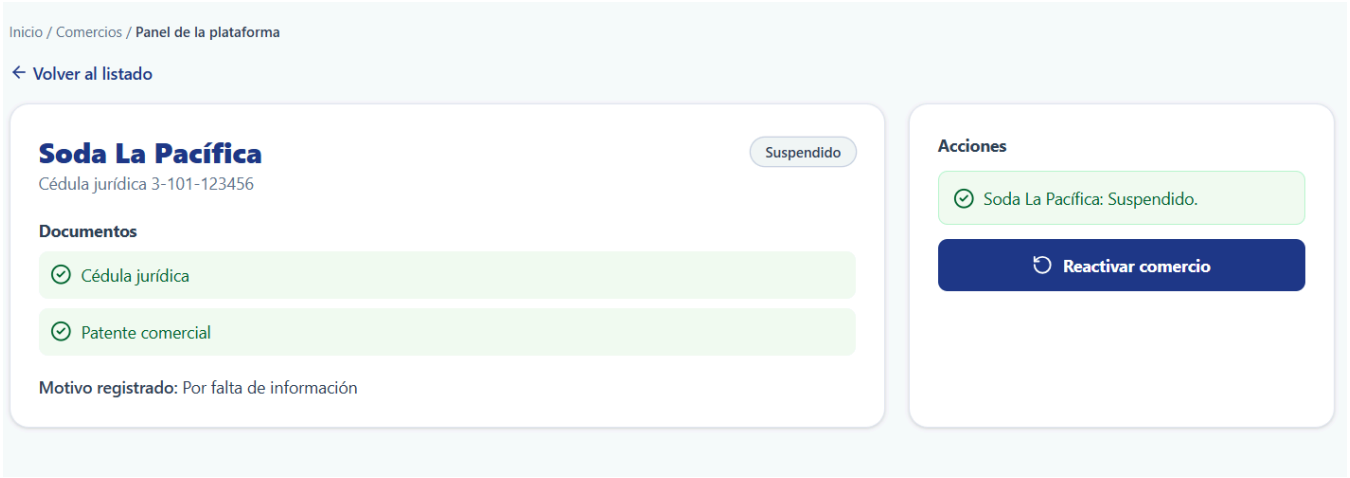
5.Solicitud aprobada



6. Proceso para suspender comercio



7.Comercio suspendido.



9. Dobles de prueba de las dependencias externas

Permiten avanzar sin esperar a los otros subgrupos: sustituyen a cada módulo o servicio externo del que depende el módulo.

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M4 — evento AlertaStockMinimo	M4 Inventario v1.2.0 / evento AlertaStockMinimo v1.0.0	Stub	Recepción correcta de la alerta con id_comercio, id_variacion, id_ubicacion, cantidad_disponible, stock_minimo, fecha_evento, id_evento y version_evento; prueba de evento duplicado mediante id_evento.	app/Services/M2_Comercios/InventarioStub.php	Cuando el evento AlertaStockMinimo de M4 esté ratificado y disponible en el ambiente de integración.
M5 — obtenerSubpedido	PedidosServiceInterface v1.1.0	Fake	Consulta exitosa de un subpedido con id_pedido, id_comercio, subtotal, costo de envío, impuesto, estado y líneas; subpedido inexistente/no accesible.	app/Services/M2_Comercios/PedidosFake.php	Cuando la implementación real de PedidosServiceInterface v1.1.0 esté disponible en el ambiente de integración.
M5 — evento SubpedidoDespachado	M5 Pedidos v1.1.0 / evento SubpedidoDespachado v1.0.0	Stub	Evento válido que dispara el cálculo de comisión; repetición del mismo id_evento para comprobar que M2 no duplique la comisión; payload incompleto o inválido.	app/Services/M2_Comercios/SubpedidoDespachadoStub.php	Cuando M5 publique y entregue SubpedidoDespachado v1.0.0 desde la implementación real.

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M7 — obtenerResumenFinancieroComercio	M7 ERP Contable v1.0.0	Stub	Respuesta correcta con ingresos, costos, gastos y utilidad para un período; parámetros de fecha inválidos; acceso sin autenticación o sin permisos.	app/Services/M2_Comercios/ContabilidadStub.php	Cuando el contrato y la implementación de M7 v1.0.0 estén disponibles en el ambiente de integración.